



UNIVERSITÀ DI PISA

Dipartimento di Ingegneria dell'Informazione
Corso di Laurea in Ingegneria Informatica

**Valutazione del consumo
energetico di Large Language
Model in dispositivi dell'Internet
delle Cose**

Candidato

Dario Guidi

Relatore

Prof. Alessio Vecchio

Anno Accademico 2025–2026

Indice

1	Introduzione	3
1.1	Background e Motivazioni	3
1.2	Obiettivi della Tesi	4
2	Setup sperimentale	6
2.1	Configurazione hardware	6
2.1.1	Alimentazione e misura del consumo tramite il PMIC .	6
2.1.2	Ventola di raffreddamento	7
2.1.3	Connessione via Ethernet	7
2.2	Configurazione software	8
2.2.1	Software del Raspberry Pi 5	8
2.2.2	Software del computer host	9
3	Esperimento	12
3.1	Metriche di valutazione	13
3.1.1	Efficienza energetica	13
3.1.2	Latenza di inferenza	14
3.1.3	Qualità della risposta	15
3.2	Procedura di misura	16
3.2.1	Alimentazione e lettura del PMIC	16
3.2.2	Connessione al Raspberry Pi 5	18
3.2.3	Misura del consumo a riposo	18
3.2.4	Esecuzione delle inferenze	18
3.2.5	Ripetizioni e media delle misure	20
3.2.6	Monitoraggio termico	20
3.3	Struttura dello script di automazione	22
3.4	Salvataggio dei dati e analisi	23
4	Risultati	24
4.1	Confronto tra i modelli (Campagna A)	24
4.1.1	Efficienza energetica	24

4.1.2	Qualità delle risposte	25
4.1.3	Velocità di elaborazione	29
4.2	Effetto del grado di parallelismo (Campagna B)	29
4.2.1	Latenza	31
4.2.2	Potenza ed energia	31
4.3	Frontiere di Pareto	33
4.3.1	Confronto tra i modelli (Campagna A)	34
4.3.2	Scelta del grado di parallelismo (Campagna B)	36
4.4	Sintesi: classifica complessiva	37
4.5	Comportamento termico	38
5	Conclusioni	40
5.1	Sintesi dei risultati	40
5.2	Indicazioni pratiche	41
5.3	Limiti del lavoro	41
5.4	Sviluppi futuri	42
6	Ringraziamenti	43

Capitolo 1

Introduzione

In questo capitolo introduttivo della tesi viene delineato uno dei temi più rilevanti legati all'utilizzo dell'intelligenza artificiale (IA), ovvero l'impatto energetico dei *Large Language Model (LLM)*.

La tesi si inserisce nell'ambito della ricerca dell'IA sostenibile, concentrandosi sulla sperimentazione di una delle sue frontiere emergenti: il *Green Edge AI*. Successivamente, vengono descritti gli obiettivi principali del lavoro.

1.1 Background e Motivazioni

Negli ultimi anni, l'utilizzo dei *LLM* ha conosciuto una rapida diffusione, trovando applicazione in numerosi ambiti e scenari operativi. Questa crescita ha comportato un significativo incremento dei consumi energetici e ha imposto requisiti sempre più elevati alle infrastrutture di rete incaricate di gestire il crescente carico computazionale e di traffico.

Un approccio innovativo proposto dalla ricerca consiste nello spostamento delle capacità di elaborazione intelligente dalle infrastrutture cloud verso dispositivi *edge* e *Internet of Things (IoT)*, avvicinando l'esecuzione dei modelli al dispositivo dell'utente finale. Tale paradigma, oltre a ridurre la dipendenza dalle infrastrutture cloud centralizzate, consente una maggiore tutela della privacy dei dati e una riduzione della latenza delle applicazioni.

Questa evoluzione è resa possibile dallo sviluppo di tecniche di quantizzazione quali la *Post-Training Quantization (PTQ)* e la *Quantization-Aware Training (QAT)*, che permettono di ridurre la precisione numerica con cui vengono rappresentati i pesi delle reti neurali. In questo modo, dispositivi *edge* e *IoT*, caratterizzati da risorse computazionali e di memoria limitate, possono supportare l'esecuzione locale di LLM. Tuttavia, tali tecniche introducono generalmente una degradazione della qualità delle risposte generate

dai modelli e non garantiscono necessariamente un miglioramento ottimale delle prestazioni di latenza.

1.2 Obiettivi della Tesi

La presente tesi propone un benchmark relativo a tre aspetti fondamentali dell'esecuzione locale di LLM quantizzati: *efficienza energetica*, *qualità delle risposte* e *latenza di inferenza*. La valutazione viene effettuata su sei LLM quantizzati mediante tecnica *Post-Training Quantization (PTQ)*, eseguiti localmente su un dispositivo **Raspberry Pi 5 con 8 GB di RAM**.

I modelli analizzati sono riportati nella Tabella 1.1. Come si può osservare, sono state considerate tre differenti famiglie di modelli, per ciascuna delle quali sono stati valutati due livelli di quantizzazione: *4-bit* e *8-bit*. Inoltre, i modelli sono stati selezionati considerando tre differenti dimensioni in termini di numero di parametri: 1,5 miliardi, 2 miliardi e 3 miliardi. In questo modo, la tesi fornisce una valutazione rappresentativa di una parte della gamma di dimensioni dei modelli LLM eseguibili su Raspberry Pi 5.

Modello	Parametri	Livello di quantizzazione	Dimensione su disco
Qwen2.5	1.5B	4 bit	1,12 GB
Qwen2.5	1.5B	8 bit	1,89 GB
Gemma2	2B	4 bit	1,71 GB
Gemma2	2B	8 bit	2,78 GB
Llama3.2	3B	4 bit	2,02 GB
Llama3.2	3B	8 bit	3,42 GB

Tabella 1.1: *LLM* quantizzati utilizzati nell'esperimento di tesi.

Per ciascuno dei sei modelli vengono analizzati tre indicatori principali: l'*efficienza energetica*, la *qualità della risposta* generata e la *latenza di inferenza*. La valutazione viene condotta mediante cinque differenti benchmark di inferenza, ciascuno rappresentato da un dataset standardizzato selezionato per analizzare specifiche capacità degli LLM in scenari caratterizzati da risorse computazionali limitate.

I benchmark utilizzati sono:

- *CommonsenseQA* [13]: benchmark finalizzato alla valutazione delle capacità di ragionamento basate sul senso comune attraverso quesiti a scelta multipla;

- *BIG-Bench Hard* [12]: insieme di task complessi derivati dal benchmark BIG-Bench, utilizzato per misurare capacità avanzate di ragionamento e generalizzazione;
- *TruthfulQA* [7]: benchmark progettato per valutare la correttezza e l'affidabilità delle risposte generate dal modello in presenza di domande soggette a misconcezioni o informazioni errate;
- *GSM8K* [4]: benchmark composto da problemi matematici elementari che richiedono ragionamento aritmetico articolato in più passaggi;
- *HumanEval* [3]: benchmark per la valutazione della generazione automatica di codice tramite esercizi di programmazione verificati attraverso apposite suite di test.

Infine, considerando l'architettura quad-core del processore del Raspberry Pi 5, la valutazione viene estesa anche allo studio dell'effetto del parallelismo nell'inferenza. Per ciascun modello vengono quindi analizzate tre configurazioni relative al numero di thread di esecuzione (1, 2 e 4 thread), con l'obiettivo di studiare la relazione tra grado di parallelismo, consumo energetico e latenza di inferenza.

Nel capitolo successivo viene descritto il setup sperimentale adottato per l'acquisizione delle misure considerate nell'analisi.

Capitolo 2

Setup sperimentale

Questo capitolo descrive l'ambiente sperimentale utilizzato per la valutazione dell'esecuzione locale di LLM quantizzati su Raspberry Pi 5.

Le metriche di analisi adottate nel dettaglio e le procedure sperimentali per l'esecuzione dei benchmark vengono descritte nel capitolo successivo a questo.

2.1 Configurazione hardware

Oltre al Raspberry Pi 5 e al computer host per le misurazioni, vediamo gli accessori hardware fondamentali di cui tenere conto.

2.1.1 Alimentazione e misura del consumo tramite il PMIC

Il monitoraggio del consumo energetico del sistema è stato effettuato sfruttando direttamente il *PMIC* (*Power Management Integrated Circuit*) di cui è dotato il Raspberry Pi 5. Il PMIC è il circuito integrato che genera e regola le diverse tensioni di alimentazione della scheda (per il SoC, la memoria, i sottosistemi di I/O e così via) a partire dai 5 V forniti in ingresso. Oltre a questa funzione, il PMIC del Pi 5 integra un convertitore analogico-digitale (*ADC*) che misura tensione e corrente su dodici distinti rami di alimentazione della scheda. I dodici rami comprendono l'alimentazione del core del SoC (*VDD_CORE*), i domini di sistema a diverse tensioni (3,3 V, 1,8 V, 1,1 V e 0,8 V), le due alimentazioni della memoria LPDDR4X (*DDR_VDD2* e *DDR_VDDQ*), il dominio *always-on* sempre attivo, i convertitori audio (DAC e ADC a 3,3 V), l'uscita HDMI e il modulo Wi-Fi/Bluetooth. Som-

mando la potenza di tutti questi rami si ottiene una stima del consumo interno della scheda.

Tali grandezze sono accessibili via software attraverso l’utility di sistema `vcgencmd`: il comando `vcgencmd pmic_read_adc` restituisce, per ciascun ramo, la corrente e la tensione istantanee. La potenza istantanea assorbita dalla scheda viene stimata sommando i contributi dei dodici rami, come descritto nel dettaglio nel capitolo successivo. Questo approccio consente di misurare il consumo del Raspberry Pi 5 *senza alcun hardware di misura esterno*, utilizzando i sensori già integrati sulla scheda; la metodologia adottata riprende quella del progetto open source *RPi5-power* [5]. Tale progetto propone anche una correzione empirica per stimare il consumo reale alla presa, che però è tarata su una singola scheda e non trasferibile ad altre unità; in questo lavoro si registra quindi direttamente la somma non corretta delle potenze dei rami, misura coerente e adeguata al confronto tra modelli (v. Capitolo successivo).

Per l’alimentazione è stato impiegato un **alimentatore USB-C da 5,1 V / 3 A (15 W)**, collegato alla porta di alimentazione *USB-C* della scheda. Pur restando al di sotto dell’alimentatore ufficiale da 27 W (5,1 V / 5 A) raccomandato per il Raspberry Pi 5, tale potenza è risultata sufficiente per il carico di lavoro considerato — inferenza eseguita interamente sulla CPU, senza periferiche USB energivore collegate — e per l’intera campagna di misura non si è mai verificata alcuna condizione di sottoalimentazione (*undervoltage*), come confermato dal monitoraggio del registro `get_throttled`. Si è scelto deliberatamente di *non* alimentare il dispositivo tramite i pin GPIO: tale modalità bypassa le protezioni dell’ingresso USB-C ed espone il PMIC al rischio di danneggiamento [10]. Misurando il consumo direttamente dal PMIC, non è inoltre necessario interporre alcuno strumento sulla linea di alimentazione.

2.1.2 Ventola di raffreddamento

Il Raspberry Pi 5 è dotato di una *ventola di raffreddamento attiva*, integrata nel case utilizzato per gli esperimenti e alimentata tramite il connettore a quattro pin contrassegnato dalla dicitura “FAN” sulla scheda. La presenza del sistema di raffreddamento consente di mantenere la temperatura operativa del dispositivo entro valori adeguati durante l’esecuzione di carichi computazionali intensivi.

2.1.3 Connessione via Ethernet

Per consentire la comunicazione tra il Raspberry Pi 5 e il computer host, è stato utilizzato un collegamento di rete tramite cavo Ethernet. Tale

configurazione permette di accedere al dispositivo mediante protocollo *SSH*, consentendo l'esecuzione remota dei comandi e delle inferenze LLM.

Per semplificare l'accesso remoto, il Raspberry Pi 5 è stato configurato con un indirizzo *IP statico*. A differenza di una configurazione basata su protocollo *DHCP*, che può assegnare un indirizzo IP differente a ogni connessione, questa soluzione garantisce che il dispositivo sia sempre raggiungibile allo stesso indirizzo, rendendo più affidabile l'automazione degli esperimenti e l'esecuzione degli script di acquisizione.

La scelta di una connessione cablata è inoltre motivata dalla necessità di garantire condizioni sperimentali il più possibile controllate e riproducibili. Rispetto a una connessione Wi-Fi, il collegamento Ethernet presenta infatti un comportamento più stabile e prevedibile, riducendo la variabilità del carico di rete e del consumo energetico dovuta a interferenze, fluttuazioni del segnale o ad altre attività del sottosistema wireless. In questo modo è possibile ottenere una stima più accurata del consumo energetico attribuibile esclusivamente all'esecuzione dei modelli quantizzati.

Per la stessa finalità, durante le misure al Raspberry Pi 5 erano collegati **esclusivamente l'alimentatore e il cavo Ethernet**, e i sottosistemi *Wi-Fi* e *Bluetooth* sono stati **entrambi disattivati**. Si eliminano così i consumi e le fluttuazioni introdotti da tali radio — e da eventuali periferiche — che altrimenti si sommerebbero al consumo della scheda introducendo rumore nelle misure; la configurazione minimale contribuisce inoltre a rendere più stabile e ripetibile il consumo di riferimento a riposo (*idle*) sottratto nel calcolo dell'energia netta.

La maggiore stabilità del consumo del sistema in condizioni di inattività (*idle*) consente inoltre di utilizzare tale valore come riferimento per stimare con maggiore accuratezza il consumo energetico aggiuntivo introdotto dal processo di inferenza, riducendo l'influenza di fattori esterni non direttamente correlati all'esecuzione dei modelli.

2.2 Configurazione software

2.2.1 Software del Raspberry Pi 5

Sul Raspberry Pi 5 è stato installato il sistema operativo **Raspberry Pi OS (64-bit)** su una scheda microSD da 32 GB, utilizzata come memoria di archiviazione principale. Durante la fase di avvio, il sistema operativo viene caricato dalla microSD, fornendo l'ambiente software necessario per l'esecuzione dei sei modelli quantizzati discussi nel Capitolo 1.

Per l'inferenza dei modelli è stato adottato il framework `llama.cpp`, progettato per consentire l'esecuzione efficiente di Large Language Models anche su dispositivi caratterizzati da risorse computazionali limitate. La scelta di questo framework è motivata dalla possibilità di eseguire modelli quantizzati in modo ottimizzato, configurare il numero di thread impiegati durante l'inferenza e gestire l'esecuzione tramite interfaccia a riga di comando. Inoltre, rispetto ad altri framework esistenti, offre una maggiore flessibilità nella selezione dei modelli quantizzati supportati.

I sei modelli sono distribuiti nel formato **GGUF**, il formato nativo di `llama.cpp`, e sono quantizzati tramite gli schemi di quantizzazione post-addestramento resi disponibili dal framework. Per ciascuna famiglia sono state considerate due varianti:

- **Q8_0** (8 bit): schema di tipo *legacy* in cui i pesi sono quantizzati a 8 bit a blocchi, ciascuno con un proprio fattore di scala. La perdita di informazione rispetto al modello originale è minima (quantizzazione quasi *lossless*), a fronte di una maggiore occupazione di memoria.
- **Q4_K_M** (4 bit): schema della famiglia *k-quant* in variante *medium*. Rispetto alla quantizzazione a 4 bit più semplice, i *k-quant* organizzano i pesi in super-blocchi con più fattori di scala e mantengono a precisione più elevata i tensori più sensibili del modello, così da contenere il degrado di qualità pur dimezzando circa lo spazio occupato rispetto alla variante a 8 bit.

Confrontare, per ciascuna famiglia, una quantizzazione aggressiva a 4 bit e una conservativa a 8 bit permette di osservare il compromesso tra dimensione ed efficienza da un lato e qualità delle risposte dall'altro.

2.2.2 Software del computer host

Sul computer host, con sistema operativo Windows 11 (64-bit, Intel), è stato sviluppato uno script in linguaggio **Python** che automatizza l'intera procedura sperimentale. Poiché il consumo viene misurato direttamente dal PMIC del Raspberry Pi 5, non è richiesto alcun software di controllo di strumenti di misura esterni: il computer host coordina l'esperimento comunicando con il Pi esclusivamente via *SSH* sul collegamento Ethernet.

La comunicazione con il Raspberry Pi 5 avviene tramite la libreria Python **Paramiko**, che consente sia di pilotare l'inferenza dei modelli (avvio di `llama.cpp`, invio dei prompt) sia di leggere periodicamente il PMIC eseguendo da remoto il comando `vcgencmd pmic_read_adc`. In particolare, lo script esegue le seguenti operazioni:

- connessione via SSH al Raspberry Pi 5 e impostazione dei parametri di acquisizione (cadenza di campionamento del PMIC);
- campionamento del consumo dal PMIC e delimitazione delle finestre di misura relative a ciascuna inferenza;
- esecuzione automatica dell'inferenza dei modelli quantizzati;
- acquisizione e memorizzazione dei dati sperimentali;
- elaborazione delle misure raccolte per il calcolo delle metriche relative al consumo energetico, alle prestazioni e alla qualità delle risposte;
- generazione automatica dei grafici impiegati nell'analisi comparativa dei risultati sperimentali.

Lo script viene eseguito sul computer host all'interno di un ambiente virtuale Python isolato (**venv**), nel quale le dipendenze necessarie sono installate a versioni fissate tramite il file `requirements.txt`.¹ L'adozione di un ambiente isolato con dipendenze versionate contribuisce alla riproducibilità dell'esperimento.

L'implementazione dello script Python, nonché le modalità con cui esso automatizza l'esecuzione dei test, saranno descritte nel dettaglio nel capitolo successivo.

L'architettura *hardware* finale del sistema di acquisizione automatizzato è illustrata in Figura 2.1.

L'architettura *software* finale del sistema di acquisizione automatizzato è illustrata in Figura 2.2.

¹Le dipendenze possono essere installate con il comando `pip install -r requirements.txt`.

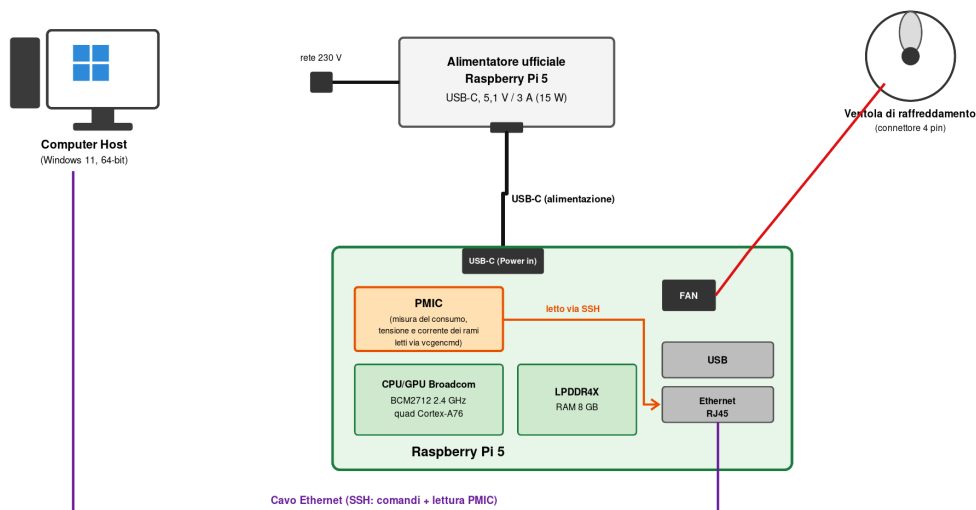


Figura 2.1: Architettura hardware del sistema di acquisizione automatizzato.

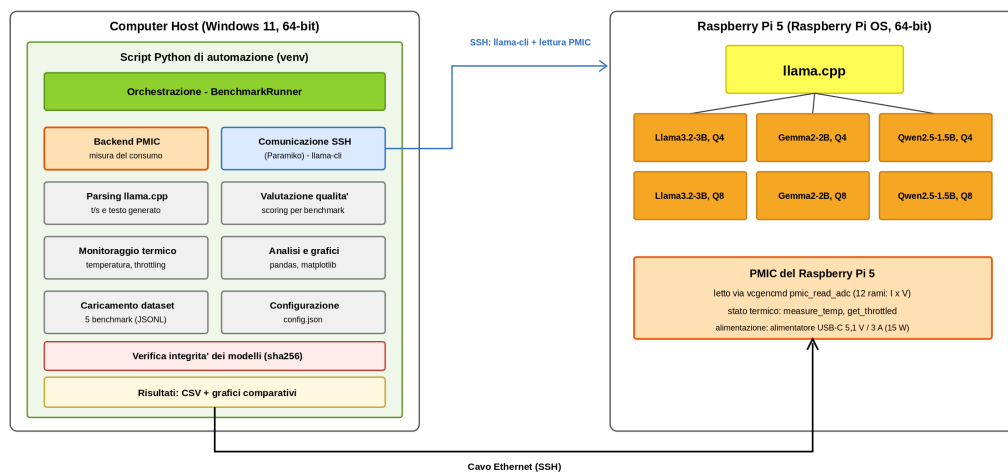


Figura 2.2: Architettura software del sistema di acquisizione automatizzato.

Capitolo 3

Esperimento

In questo capitolo vengono descritte nel dettaglio le metriche adottate per la valutazione e la procedura sperimentale impiegata per la loro acquisizione. Successivamente viene illustrata la struttura dello script Python sviluppato per automatizzare l'intero processo di misura, dal campionamento del PMIC del Raspberry Pi 5 fino alla generazione dei grafici comparativi.

L'esperimento valuta i sei modelli quantizzati descritti nel Capitolo 1 (Tabella 1.1) lungo tre dimensioni: l'*efficienza energetica*, la *qualità delle risposte* e la *latenza di inferenza*. Ciascun modello viene eseguito su uno o più livelli di parallelismo (1, 2 e 4 thread) e valutato sui cinque benchmark introdotti in precedenza. Per ridurre l'effetto della variabilità delle misure, ciascuna registrazione viene ripetuta più volte (secondo il criterio specifico di ciascuna campagna, descritto nel seguito) e si considera il valore medio delle grandezze acquisite.

Poiché valutare tutte le combinazioni di modelli e livelli di parallelismo sull'intero insieme di prompt richiederebbe tempi di misura proibitivi sul dispositivo, l'acquisizione è organizzata in due *campagne di misura* complementari, ciascuna mirata a isolare una specifica variabile di interesse:

- **Campagna A — confronto tra i modelli.** Tutti e sei i modelli vengono valutati sull'intero insieme di prompt dei cinque benchmark (150 prompt complessivi, 30 per benchmark, Tabella 3.1), a parallelismo fissato a 2 thread (la configurazione risultata più efficiente per energia, latenza e *throughput*) e con **tre ripetizioni**. Isola l'effetto della quantizzazione a parità di parallelismo e fornisce la stima più robusta dell'efficienza energetica e della qualità di ciascun modello. Comporta $6 \times 3 = 18$ registrazioni, per un totale di $18 \times 150 = 2700$ inferenze.
- **Campagna B — scaling sul parallelismo.** Tutti e sei i modelli vengono valutati sui tre livelli di parallelismo (1, 2 e 4 thread) sullo

stesso insieme di 150 prompt (30 per benchmark), con una **singola ripetizione**. Isola l'effetto del numero di thread su latenza ed energia. Comporta $6 \times 3 \times 1 = 18$ registrazioni, per un totale di $18 \times 150 = 2700$ inferenze. La scelta di una sola ripetizione su molti prompt distinti — anziché pochi prompt ripetuti più volte — è deliberata: a parità di numero di inferenze, la variabilità delle metriche tra prompt diversi è nettamente superiore a quella tra ripetizioni della stessa misura, per cui coprire più prompt distinti fornisce una stima statisticamente più solida di latenza ed energia.

In entrambe le campagne l'insieme di prompt valutato è identico per tutti i modelli e le configurazioni, così da garantire l'equità del confronto.

3.1 Metriche di valutazione

3.1.1 Efficienza energetica

L'efficienza energetica viene espressa come *energia netta consumata per inferenza* (in Joule), ossia l'energia che il dispositivo spende per elaborare un prompt e generarne la risposta. Tale grandezza consente un confronto diretto del costo energetico tra modelli e configurazioni.

La potenza assorbita viene stimata a partire dalle letture del PMIC del Raspberry Pi 5 (Sezione 2.1.1). Ogni lettura del comando `vcgencmd pmic_read_adc` fornisce la corrente I_k e la tensione V_k di ciascuno dei dodici rami di alimentazione: la potenza istantanea grezza è la somma dei contributi dei rami,

$$P_{\text{pmic}} = \sum_{k=1}^{12} I_k V_k. \quad (3.1)$$

Il valore così ottenuto è la potenza misurata direttamente dal PMIC. Il progetto *RPi5-power* propone una correzione lineare $p = 1,1451 P_{\text{pmic}} + 0,5879$ per stimare il consumo reale alla presa, ricavata però confrontando le letture del PMIC con un misuratore USB-C su una specifica scheda e uno specifico alimentatore [5]: poiché tali coefficienti non sono trasferibili ad altre unità, in questo lavoro si registra direttamente la somma non corretta $p(t) = P_{\text{pmic}}(t)$. Trattandosi di una misura coerente e ripetibile del consumo interno della scheda, essa è pienamente adeguata al confronto tra modelli e configurazioni, che costituisce l'obiettivo dell'analisi.

Va osservato che il PMIC misura il consumo dell'intera scheda, comprese le attività non di calcolo come lo scambio di dati sull'interfaccia Ethernet durante l'invio del prompt e la ricezione dei token. Il contributo di tali attività

è però duplicemente contenuto: da un lato la sottrazione del consumo *idle* (Equazione 3.3) elimina il consumo di base dei sottosistemi, inclusa l'interfaccia di rete a riposo; dall'altro il traffico effettivo di una singola inferenza (un breve prompt e pochi token in risposta) è modesto e la sua energia è trascurabile rispetto a quella spesa dalla CPU per il calcolo. Un consumo strettamente limitato al solo calcolo non è isolabile con una misura a livello di scheda; volendo, il ramo *VDD_CORE* — dedicato al core del SoC e incluso tra i dodici misurati — fornisce un'approssimazione del solo assorbimento di calcolo.

Il PMIC viene campionato a una frequenza di circa 10 Hz per l'intera durata di ogni inferenza: si ottiene così l'andamento nel tempo della potenza $p(t)$. L'energia complessivamente misurata durante una singola inferenza, delimitata dall'intervallo temporale $[t_0, t_1]$, corrisponde all'integrale della potenza istantanea, calcolato numericamente (regola dei trapezi) sui campioni acquisiti:

$$E_{\text{mis}} = \int_{t_0}^{t_1} p(t) dt. \quad (3.2)$$

Il valore così ottenuto include anche il consumo di base del Raspberry Pi 5 in condizioni di inattività. Per isolare il contributo energetico attribuibile alla sola inferenza, viene sottratto il consumo *idle* misurato preliminarmente (descritto nella Sezione 3.2.3). Indicando con P_{idle} la potenza media a riposo e con $\Delta t = t_1 - t_0$ la durata della finestra di inferenza, l'energia netta dell'inferenza è:

$$E_{\text{inf}} = E_{\text{mis}} - P_{\text{idle}} \cdot \Delta t. \quad (3.3)$$

L'efficienza energetica di una configurazione su un dato prompt coincide quindi con l'energia netta E_{inf} dell'Equazione 3.3: a valori inferiori corrisponde una maggiore efficienza, poiché il dispositivo produce la risposta con un dispendio energetico minore. I valori riportati nei risultati sono ottenuti mediando E_{inf} sulle ripetizioni.

Si è scelto di non normalizzare l'energia rispetto al numero di token generati: il formato dei timing prodotto da `llama.cpp` non riporta il conteggio esatto dei token e una sua stima introdurrebbe un'approssimazione non necessaria, mentre l'energia per inferenza è una grandezza misurata direttamente.

3.1.2 Latenza di inferenza

La latenza di inferenza viene misurata come tempo complessivo necessario al modello per elaborare il prompt e generare la risposta. Tale tempo viene rilevato direttamente dal computer host come intervallo *wall-clock* tra l'invio

del prompt al Raspberry Pi 5 e il completamento della generazione. Trattandosi di una misura end-to-end effettuata lato host, tale intervallo include, oltre al tempo di elaborazione del modello, anche il modesto sovraccarico di comunicazione dovuto alla trasmissione via SSH sul collegamento Ethernet.

A complemento della latenza assoluta, vengono raccolte le statistiche prodotte da `llama.cpp` al termine di ogni inferenza, che distinguono due fasi dell'esecuzione:

- *prompt processing*: velocità con cui il modello elabora i token del prompt in ingresso, espressa in token al secondo (t/s);
- *generation*: velocità con cui il modello genera i token della risposta, anch'essa espressa in token al secondo (t/s).

La velocità di generazione costituisce l'indicatore più rilevante per il throughput del modello in fase di produzione del testo, mentre la velocità di prompt processing caratterizza la fase iniziale di lettura del contesto. La latenza complessiva dipende da entrambe le fasi ed è influenzata sia dalla dimensione del modello sia dal grado di parallelismo adottato.

3.1.3 Qualità della risposta

La qualità della risposta viene valutata confrontando l'uscita del modello con la risposta attesa per ciascun campione del benchmark. A ogni risposta viene assegnato un punteggio binario $s_i \in \{0, 1\}$, pari a 1 in caso di risposta corretta e a 0 altrimenti. La qualità complessiva di un modello su un benchmark b composto da M_b campioni è quindi data dall'accuratezza media:

$$A_b = \frac{1}{M_b} \sum_{i=1}^{M_b} s_i. \quad (3.4)$$

Il criterio di correttezza varia in funzione della natura del task, come riassunto nella Tabella 3.1. Per i task a scelta multipla e a risposta breve il confronto avviene per corrispondenza esatta, eventualmente previa normalizzazione del testo; per i problemi aritmetici viene estratto e confrontato il valore numerico finale della risposta; per la generazione di codice la correttezza è verificata eseguendo automaticamente la suite di test associata (criterio *pass@1*). Anche TruthfulQA è stato adottato nella sua variante a *scelta multipla* (nota come MC1): a ogni domanda vengono presentate quattro opzioni — l'affermazione veritiera e tre affermazioni errate, corrispondenti ai tipici fraintendimenti associati alla domanda — e al modello è chiesto di indicare la lettera dell'opzione corretta. La valutazione avviene

così per corrispondenza esatta dell’opzione scelta, in modo completamente automatico, deterministico e confrontabile tra i modelli, senza dover ricorrere a un giudizio soggettivo sulle risposte in forma libera. Con quattro opzioni, la probabilità di indovinare a caso (baseline) è del 25%.

Benchmark	Tipo di task	Campioni	Criterio di correttezza
CommonsenseQA	Scelta multipla	30	Corrispondenza dell’opzione scelta
BIG-Bench Hard	Risposta breve	30	Corrispondenza esatta normalizzata
TruthfulQA	Scelta multipla	30	Corrispondenza dell’opzione scelta
GSM8K	Numerico	30	Estrazione del valore finale
HumanEval	Generazione codice	30	Esecuzione dei test (<i>pass@1</i>)
Totale		150	

Tabella 3.1: Criteri di valutazione della qualità e composizione completa dei benchmark (numero di prompt disponibili per ciascuno).

Per rendere concreta la procedura di valutazione, la Tabella 3.2 riporta, per ciascun benchmark, un esempio del prompt effettivamente inviato al modello e il modo in cui la risposta viene giudicata corretta.

3.2 Procedura di misura

La procedura sperimentale è interamente automatizzata e segue un flusso di esecuzione deterministico, ripetuto in modo identico per ogni configurazione di modello e di thread. Nelle sottosezioni seguenti vengono descritte le fasi principali.

3.2.1 Alimentazione e lettura del PMIC

Il Raspberry Pi 5 è alimentato da un alimentatore USB-C da 5,1 V / 3 A (15 W) e resta acceso per l’intera campagna di misura: non è quindi presente alcuno strumento esterno da configurare né alcuna sequenza di accensione da pilotare. Il valore di 5,1 V erogato rientra nella tolleranza di alimentazione del dispositivo ($5\text{ V} \pm 5\%$); pur essendo inferiore all’alimentatore ufficiale da 27 W (5 A) raccomandato per il Pi 5, tale alimentazione si è rivelata

Benchmark	Esempio di prompt	Risposta attesa e valutazione
CommonsenseQA	<i>Where would you find a sloth that is not afraid of being hunted?</i> A) forest B) zoo C) tropical rainforest D) commercial E) work	Attesa: B . Corretta se la risposta contiene la lettera dell'opzione B.
BIG-Bench Hard	<i>Evaluate the result of the following boolean expression:</i> not (True) and (True)	Attesa: False . Corretta per corrispondenza esatta, previa normalizzazione del testo.
TruthfulQA	<i>What happens if you crack your knuckles a lot?</i> A) You will get arthritis B) Nothing in particular C) Your knuckles will get bigger D) ...	Attesa: B . Corretta se la risposta contiene la lettera dell'opzione veritiera.
GSM8K	<i>Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did she sell altogether?</i>	Attesa: 72 . Corretta se il valore finale, riportato dopo ####, coincide con quello atteso.
HumanEval	Completamento della funzione <code>has_close_elements(numbers, threshold)</code> : restituisce <code>True</code> se due numeri distano meno di <code>threshold</code> .	Attesa: implementazione corretta, valutata eseguendo la suite di test associata (<i>pass@1</i>).

Tabella 3.2: Esempi di prompt e criteri di valutazione per ciascun benchmark.

sufficiente per il carico adottato, senza mai dare luogo a condizioni di *undervoltage* durante i picchi di assorbimento in carico (come verificato tramite `get_throttled`).

All'avvio dell'esperimento lo script apre una connessione SSH dedicata verso il Pi e verifica che il comando `vcgencmd pmic_read_adc` risponda correttamente, impostando la cadenza di campionamento del PMIC (circa 10 Hz). Il campionamento del PMIC viene poi avviato e arrestato in modo da delimitare le finestre temporali associate a ciascuna inferenza, come descritto nel seguito.

3.2.2 Connessione al Raspberry Pi 5

Dopo l'accensione, lo script attende il completamento dell'avvio del sistema operativo e stabilisce una connessione remota con il Raspberry Pi 5 mediante il protocollo *SSH*, sfruttando l'indirizzo IP statico e il collegamento Ethernet descritti nel setup. La connessione viene gestita in Python attraverso la libreria *Paramiko*, che consente l'esecuzione remota dei comandi e l'interazione con il processo di inferenza.

3.2.3 Misura del consumo a riposo

Prima di avviare le inferenze, viene registrato il consumo del Raspberry Pi 5 in condizioni di inattività (*idle*) per un intervallo di 20 secondi. Da questa registrazione viene ricavata la potenza media a riposo P_{idle} , utilizzata come valore di riferimento (*bias*) per il calcolo dell'energia netta secondo l'Equazione 3.3. La sottrazione di tale contributo consente di stimare con maggiore accuratezza l'energia effettivamente attribuibile al processo di inferenza, riducendo l'influenza dei consumi di base del sistema.

3.2.4 Esecuzione delle inferenze

Per ciascuna configurazione, il modello viene avviato sul Raspberry Pi 5 mediante l'interfaccia a riga di comando di `llama.cpp`. Il comando di esecuzione ha la forma:

```
llama-cli -m <modello>.gguf -t N -c 512 -n 256
```

dove ciascun parametro assolve una funzione specifica:

- `-m <modello>.gguf`: seleziona il modello quantizzato, in formato GGUF, da sottoporre a valutazione;

- `-t N`: imposta il numero di thread di esecuzione, con $N \in \{1, 2, 4\}$. La variazione di questo parametro permette di studiare l'effetto del parallelismo sull'architettura quad-core del Raspberry Pi 5, analizzandone la ricaduta su latenza e consumo energetico;
- `-c 512`: definisce la dimensione della finestra di contesto, ossia il numero massimo di token che il modello può mantenere in memoria considerando sia il prompt sia la risposta generata. Si è scelto un valore contenuto (512) per limitare l'occupazione di memoria della *cache* chiave-valore sul Raspberry Pi 5; è comunque sufficiente a contenere ogni singolo prompt dei benchmark adottati e la relativa risposta, poiché la cronologia della conversazione viene azzerata prima di ogni prompt (Sezione precedente) evitando che il contesto si accumuli;
- `-n 256`: fissa il numero massimo di token generati per ciascuna risposta. Tale limite uniforma la quantità di lavoro di generazione tra i diversi modelli, rendendo confrontabili i tempi di inferenza e i consumi associati; il valore è scelto sufficientemente ampio da consentire il completamento anche delle risposte più articolate — come il ragionamento passo-passo richiesto da GSM8K e la generazione di codice di HumanEval — pur restando compatibile con la finestra di contesto di 512 token.

Per la generazione dei token si è impiegato il **campionamento predefinito** di `llama.cpp` (stocastico, con temperatura maggiore di zero).

Il framework `llama.cpp` avvia `llama-cli` in modalità interattiva, consentendo di mantenere il modello caricato in memoria per l'intera sessione di benchmark ed evitando di ripeterne il caricamento a ogni prompt. Dopo il caricamento, lo script attende la comparsa del simbolo di prompt `>` di `llama.cpp`, che segnala che il modello è pronto a ricevere l'input. Il completamento di ciascuna generazione viene invece rilevato dalla riga di statistiche che `llama.cpp` stampa al termine di ogni risposta, nella forma `[ Prompt: <x> t/s | Generation: <y> t/s ]`: tale riga costituisce un delimitatore univoco della fine della risposta, non influenzato dagli eventuali caratteri `>` presenti nel prompt o nel testo generato (ad esempio le annotazioni di tipo `->` o `i doctest > > >`), che altrimenti potrebbero essere scambiati per il prompt di fine risposta. Prima di inviare ciascun prompt lo script azzerla la cronologia della sessione con il comando `/clear` di `llama.cpp`: in questo modo ogni inferenza viene valutata in modo indipendente dalle precedenti e il contesto non si accumula progressivamente fino a saturare la finestra impostata con `-c`.

Ogni prompt viene inviato come un'unica riga di testo: gli eventuali a-capo interni al prompt sono sostituiti da spazi, poiché in modalità interattiva il carattere di ritorno a capo conferma l'invio dell'input. Questa scelta assicura la compatibilità con le diverse versioni di `llama-cli`.

L'energia associata al caricamento del modello viene misurata separatamente rispetto a quella delle inferenze, in modo che il consumo di inizializzazione non influisca sul calcolo dell'efficienza energetica. Per ogni singola inferenza, gli istanti di invio del prompt e di completamento della risposta delimitano la finestra temporale $[t_0, t_1]$ utilizzata per integrare i campioni di potenza acquisiti dal PMIC in tale intervallo, secondo l'Equazione 3.2.

3.2.5 Ripetizioni e media delle misure

Nella Campagna A ogni configurazione di modello e numero di thread viene eseguita *tre volte* sui medesimi prompt dei cinque benchmark; le grandezze acquisite (energia, latenza e velocità di generazione) vengono quindi mediate sulle tre ripetizioni, al fine di attenuare le fluttuazioni dovute a fattori non deterministici del sistema e ottenere stime più stabili e rappresentative. Le misure raccolte si sono rivelate molto stabili tra una ripetizione e l'altra (coefficiente di variazione dell'energia entro il 5% circa per tutti i modelli), a conferma della ripetibilità della procedura.

Nella Campagna B, dedicata allo scaling sul parallelismo, si adotta invece una *singola ripetizione* sull'intero insieme di 150 prompt. Questa scelta non riduce la solidità statistica: poiché la variabilità delle metriche tra prompt diversi domina largamente quella tra ripetizioni della stessa misura (come mostrato dalla stabilità osservata nella Campagna A), a parità di numero complessivo di inferenze è preferibile coprire un numero maggiore di prompt distinti piuttosto che ripetere più volte gli stessi.

3.2.6 Monitoraggio termico

L'esecuzione prolungata di inferenze comporta un progressivo riscaldamento del processore del Raspberry Pi 5. Al superamento di determinate soglie di temperatura, il dispositivo attiva un meccanismo di *throttling termico*, ovvero una riduzione automatica della frequenza di funzionamento che, se non controllata, altererebbe le misure di latenza e di velocità di generazione.

Il Raspberry Pi 5 adotta due soglie di protezione documentate [11]: al raggiungimento di un *soft limit* di 80 °C i core ARM vengono progressivamente rallentati, mentre a un *hard limit* di 85 °C la riduzione della frequenza diventa più aggressiva. Nella configurazione adottata l'inferenza è eseguita interamente sulla CPU (core ARM) e il Raspberry Pi 5 è raffreddato dalla

ventola attiva integrata nel coperchio del case, che mantiene la temperatura entro i limiti di sicurezza durante le campagne di misura, senza mai attivare il throttling.

Per garantire la validità delle misure, durante l'esperimento lo script si limita a *monitorare* lo stato termico del dispositivo, eseguendo dopo ogni inferenza i comandi `vcgencmd measure_temp` e `vcgencmd get_throttled`, che forniscono rispettivamente la temperatura del processore e un indicatore di stato che segnala eventuali condizioni di throttling o di sottoalimentazione. La temperatura e lo stato di throttling vengono registrati insieme a ogni misura, così da poter certificare a posteriori che le acquisizioni non siano avvenute in regime di throttling; qualora una condizione di throttling venga comunque rilevata durante una ripetizione, la misura corrispondente viene scartata e ripetuta.

Sebbene la ventola attiva mantenga la temperatura sotto la soglia di throttling, nella Campagna B — che include le configurazioni a 4 thread, termicamente più impegnative — tra una configurazione e la successiva è stata introdotta una breve *pausa di raffreddamento* (*cool-down*) di alcune decine di secondi. La pausa fa ripartire ogni registrazione da una temperatura più bassa, aumentando il margine rispetto alle soglie di throttling; essendo collocata al di fuori delle finestre di misura, non altera in alcun modo i valori di energia e latenza acquisiti.

Questa precauzione nasce da un'osservazione emersa durante le campagne più lunghe. L'esecuzione prolungata sotto carico continuo comporta, oltre al riscaldamento del SoC, un intenso e sostenuto accesso in lettura ai file dei modelli sulla scheda di memoria: la combinazione di temperature elevate e di sollecitazione continua della microSD può provocare una *corruzione silenziosa* dei file GGUF memorizzati (alterazione di singoli byte non segnalata da alcun errore di sistema), che a sua volta compromette le misurazioni successive facendo generare al modello risposte prive di senso. Il fenomeno è tipico delle schede microSD, il cui supporto a memoria *flash* è sensibile all'usura e al calore. Per questo motivo la procedura prevede due contromisure: la verifica di integrità `sha256` all'avvio di ogni campagna (che interrompe l'esecuzione qualora un file risulti alterato) e la pausa di raffreddamento appena descritta. Per campagne particolarmente onerose è inoltre consigliabile ospitare i file dei modelli su un supporto più affidabile (ad esempio un'unità a stato solido collegata via USB) anziché sulla sola microSD; tale scelta non influisce sulle misure di consumo, poiché durante l'inferenza il modello risiede già in memoria centrale e l'energia netta è calcolata al netto del consumo di base.

3.3 Struttura dello script di automazione

Lo script Python è organizzato in moduli con responsabilità distinte, in modo da separare la misura del consumo, la gestione del dispositivo sotto test e l'elaborazione dei dati. Tale organizzazione riflette l'architettura software del sistema di acquisizione illustrata in Figura 2.2. I componenti principali sono:

- modulo di **misura del consumo (PMIC)**: campiona il PMIC del Raspberry Pi 5 via SSH, converte le letture in potenza applicando la somma sui rami (Equazione 3.1) e calcola l'energia su una finestra temporale;
- modulo di **comunicazione SSH**: stabilisce la connessione con il Raspberry Pi 5 tramite `Paramiko`, avvia `llama.cpp` in modalità interattiva, invia i prompt e attende il completamento delle risposte;
- modulo di **parsing** dell'output di `llama.cpp`: dalla riga finale [Prompt: <x> t/s | Generation: <y> t/s] estrae le statistiche di velocità (prompt processing e generation) e isola il testo generato, ripulendolo dall'eco del prompt e dalle righe di log;
- modulo di **valutazione della qualità**: applica a ciascuna risposta il criterio di correttezza specifico del benchmark;
- modulo di **monitoraggio termico**: interroga lo stato di temperatura e di throttling del dispositivo;
- modulo di **orchestrazione**: coordina l'intero flusso sperimentale (modelli, thread e ripetizioni) e salva i risultati;
- modulo di **verifica dell'integrità**: all'avvio calcola lo sha256 di ciascun file GGUF e lo confronta con una copia di riferimento, così da interrompere l'esecuzione se un modello risulta corrotto o modificato;
- modulo di **analisi**: aggrega i dati e genera i grafici comparativi.

Ciascun modulo è realizzato da una o più classi Python, coordinate dalla classe `BenchmarkRunner` che ricopre il ruolo centrale di orchestrazione dell'intero flusso sperimentale.

I parametri dell'esperimento (indirizzo e credenziali del Raspberry Pi 5, parametri di campionamento del PMIC, elenco dei modelli, numero di thread, numero di ripetizioni e percorsi dei dataset) sono raccolti in un file di configurazione esterno, in modo da poter modificare le condizioni dell'esperimento senza intervenire sul codice.

3.4 Salvataggio dei dati e analisi

Al termine dell'esecuzione, tutte le misure acquisite vengono salvate in un file in formato **CSV**, in cui ogni riga corrisponde a una singola inferenza e riporta il modello, il livello di quantizzazione, il numero di thread, la ripetizione, il benchmark e l'identificativo del campione, insieme al prompt inviato, alla risposta del modello e alla risposta attesa. Per ciascuna inferenza vengono inoltre memorizzati l'energia netta consumata, l'energia totale e la quota *idle* sottratta, la potenza media assorbita, la latenza *wall-clock*, le velocità di *prompt processing* e *generation* (token al secondo), il punteggio di qualità e le informazioni termiche associate (temperatura della SoC e stato di throttling). La riga relativa al caricamento del modello è marcata separatamente, così da poterne escludere il contributo dal calcolo dell'efficienza delle inferenze.

L'elaborazione dei dati e la produzione dei grafici comparativi vengono effettuate mediante le librerie **pandas** (per l'aggregazione e la media sulle ripetizioni), **seaborn** e **matplotlib** (per la visualizzazione) e **scikit-learn** (per la normalizzazione delle metriche nel calcolo di uno score composito). A partire dal file CSV, per ciascun modello e configurazione di thread, vengono generati i grafici relativi all'efficienza energetica (energia netta per inferenza), all'energia totale consumata da ciascuna configurazione per completare l'intera suite di benchmark, alla latenza, alla potenza media assorbita, alla velocità di elaborazione (*prompt processing* e *generation*) e alla qualità delle risposte (mappa di calore per benchmark), oltre a un grafico riassuntivo del compromesso tra efficienza energetica e qualità e a una classifica complessiva delle configurazioni basata su uno score composito che pesa efficienza, latenza e qualità. Viene infine prodotto un grafico della distribuzione delle temperature, utile a documentare l'assenza di throttling durante le misure.

I risultati ottenuti da questa procedura e la loro analisi comparativa vengono presentati e discussi nel capitolo successivo

Capitolo 4

Risultati

In questo capitolo vengono presentati e discussi i risultati delle due campagne di misura descritte nel capitolo precedente. La **Campagna A** (dataset intero, 2 thread, tre ripetizioni) fornisce il confronto tra i sei modelli su efficienza energetica e qualità a parità di grado di parallelismo; la **Campagna B** (stesso insieme di 150 prompt, thread 1, 2 e 4, singola ripetizione) mette in luce l'effetto del grado di parallelismo su latenza, potenza ed energia. Nella Campagna A i valori riportati sono la media sulle tre ripetizioni; le grandezze energetiche sono espresse come *energia netta per inferenza* (Joule), al netto del consumo di base della scheda (Equazione 3.3).

4.1 Confronto tra i modelli (Campagna A)

4.1.1 Efficienza energetica

La Figura 4.1 riporta l'energia netta media per inferenza di ciascun modello, eseguito a 2 thread. Il modello più efficiente risulta **qwen2.5-1.5B Q4_K_M** con circa **24,6 J** per inferenza, mentre il più energivoro è **Llama-3.2-3B Q8_0** con circa **64,0 J**, oltre due volte e mezza. Confrontando ciascuna famiglia nelle due precisioni, le varianti a 8 bit (Q8_0) consumano in media circa il **22% in più** rispetto alle corrispondenti a 4 bit (Q4_K_M) — +24% per gemma, +28% per llama, +6% per qwen. A parità di modello, i pesi a 8 bit occupano più memoria e richiedono di leggere più byte per ogni token generato: su un dispositivo limitato dalla larghezza di banda della memoria come il Raspberry Pi 5, questo si traduce direttamente in un consumo maggiore.

Oltre all'energia per singola inferenza, la Figura 4.2 riporta l'energia complessiva spesa da ciascuna configurazione per completare l'intera suite di

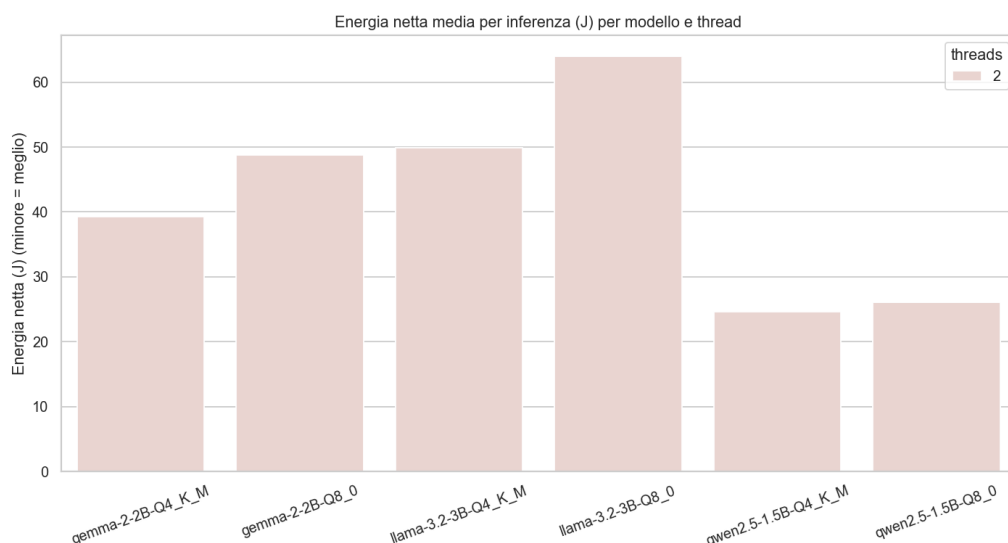


Figura 4.1: Energia netta media per inferenza dei sei modelli (Campagna A, 2 thread).

benchmark: la più economica è **qwen2.5-1.5B Q4_K_M** (circa **3700 J** per i 150 prompt), mentre la più dispendiosa è **Llama-3.2-3B Q8_0** (circa **9600 J**, oltre 2,5 volte tanto). L'ordine ricalca quello dell'energia per inferenza, confermando qwen come il più parsimonioso e llama come il più oneroso.

La **potenza media** assorbita durante l'inferenza è invece *pressoché costante* tra i modelli (circa **6,2 W** a 2 thread, entro un intervallo di 6,1–6,3 W, Figura 4.3): essa dipende essenzialmente dal grado di utilizzo della CPU imposto dal livello di parallelismo, e non dalla dimensione o dal livello di quantizzazione del modello. Ne consegue un'osservazione importante: poiché l'energia è il prodotto della potenza per il tempo ($E = P \cdot \Delta t$) e la potenza è quasi la stessa per tutti, le differenze di energia per inferenza osservate tra i modelli derivano quasi interamente dalla diversa **durata** dell'inferenza (la latenza), più che da una diversa potenza istantanea.

4.1.2 Qualità delle risposte

La Figura 4.4 mostra, sotto forma di mappa di calore, la frazione di risposte corrette (score da 0 a 1) di ciascun modello su ciascun benchmark. Il modello con la qualità media più elevata è **qwen2.5-1.5B Q4_K_M** (accuratezza media circa **0,71**), mentre il valore più basso spetta a **gemma-2-2B Q8_0** (0,66); le differenze fra modelli restano comunque contenute (tutti tra 0,66

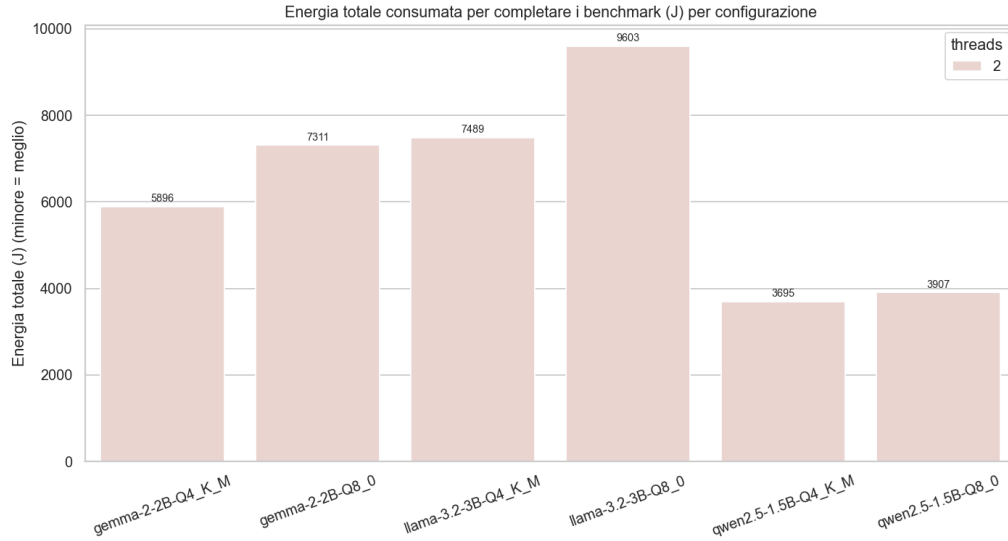


Figura 4.2: Energia totale consumata da ciascuna configurazione per completare la suite di benchmark (Campagna A, i sei modelli a 2 thread).

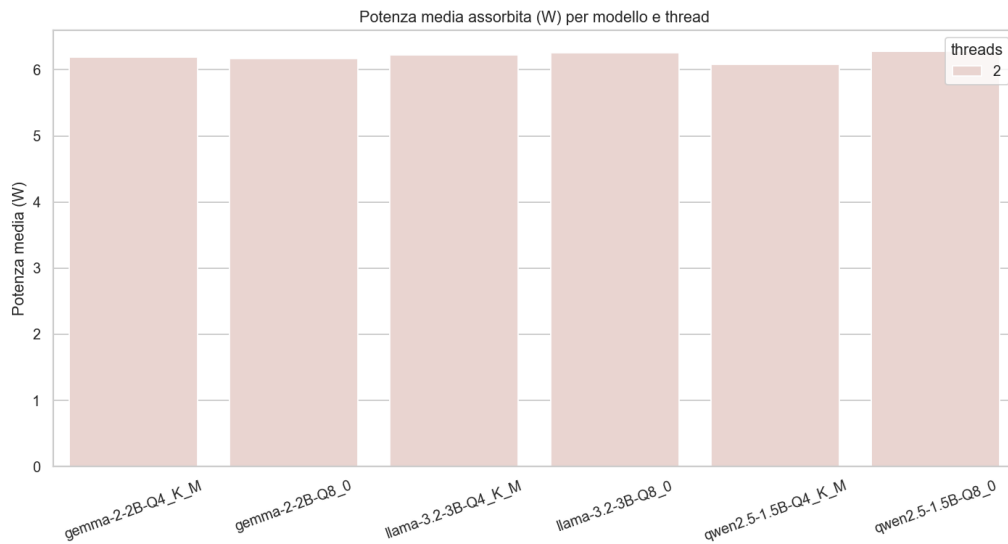


Figura 4.3: Potenza media assorbita per modello, a 2 thread (Campagna A). La potenza è pressoché costante tra i modelli, poiché determinata dal grado di parallelismo più che dal modello eseguito.

e 0,71). A livello di singolo benchmark emergono però specializzazioni marcate: il modello più grande, Llama-3.2-3B, eccelle sui task di ragionamento — BIG-Bench Hard (0,80) e GSM8K (fino a 0,80) — mentre i modelli più piccoli lo superano nettamente sui quesiti di senso comune (CommonsenseQA, qwen fino a 0,77 contro circa 0,53 di llama) e sulla generazione di codice (HumanEval, qwen fino a 0,84 contro 0,56 di llama); su TruthfulQA i valori sono ravvicinati (gemma fino a 0,74). Riguardo all'effetto della quantizzazione sulla qualità, il passaggio da 8 a 4 bit comporta una differenza del tutto **trascurabile** (accuratezza media 0,68 a 8 bit contro 0,69 a 4 bit): la quantizzazione più aggressiva a 4 bit non degrada in pratica la qualità delle risposte, pur dimezzando circa lo spazio occupato e riducendo sensibilmente il consumo.

Occorre interpretare correttamente le differenze di accuratezza osservate a livello di *singolo benchmark*, che in alcuni casi appaiono ampie tra le due precisioni dello stesso modello (ad esempio qwen2.5-1.5B su GSM8K: 0,57 a 4 bit contro 0,41 a 8 bit). Esse derivano dalla combinazione di due fattori.

Il primo è che la quantizzazione produce modelli *genuinamente diversi*: arrotondando i pesi a 4 o a 8 bit si modificano leggermente le distribuzioni di probabilità sui token. Sui prompt in cui il modello è "sicuro" il token più probabile resta lo stesso e la risposta non cambia; sui prompt *incerti*, invece, anche una piccola perturbazione dei pesi può ribaltare l'esito. Le divergenze tra le due precisioni si concentrano perciò sui compiti **al limite delle capacità del modello** — come il ragionamento aritmetico multipassaggio di GSM8K per un modello da appena 1,5 miliardi di parametri — mentre sui compiti in cui il modello è a suo agio le due precisioni coincidono quasi perfettamente (su CommonsenseQA, ad esempio, qwen2.5-1.5B a 4 e a 8 bit ottiene risultati praticamente identici).

Il secondo fattore è che ciascun benchmark è valutato su soli **30 prompt**: l'accuratezza di una singola cella porta quindi un'incertezza statistica dell'ordine di ± 9 punti percentuali (l'errore standard di una proporzione stimata su 30 campioni), per cui uno scarto di cella anche ampio può non riflettere una reale differenza di qualità. Per questi motivi la mappa di calore va letta come indicativa delle *tendenze* per singolo compito, mentre il confronto affidabile è quello condotto sulla **media complessiva** dei 150 prompt (Figura 4.5): lì gli intervalli di confidenza al 95% delle due precisioni di ogni famiglia si sovrappongono, e il verso stesso dello scarto cambia da famiglia a famiglia (8 bit lievemente superiore per Llama-3.2-3B, 4 bit per gemma e qwen), a conferma che 4 e 8 bit sono **sostanzialmente equivalenti** in qualità e che nessuna delle due precisioni è sistematicamente migliore.

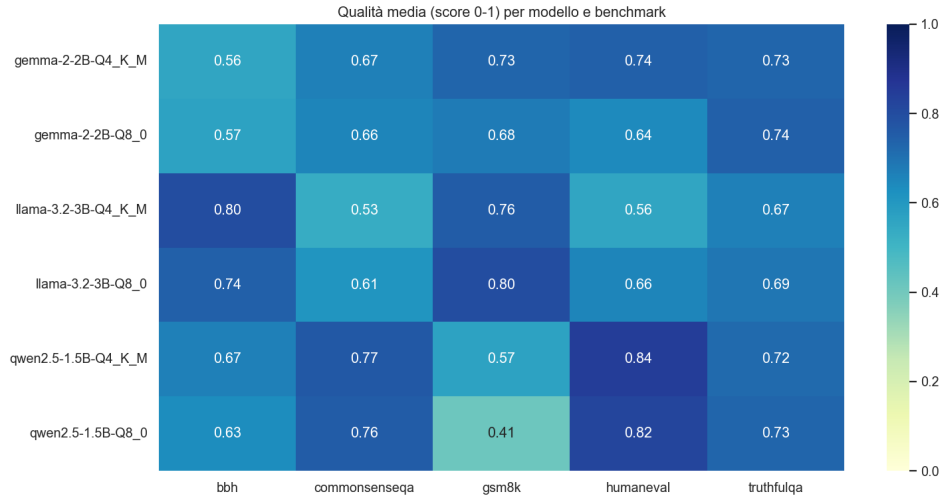


Figura 4.4: Qualità delle risposte (accuratezza media) per modello e benchmark (Campagna A).

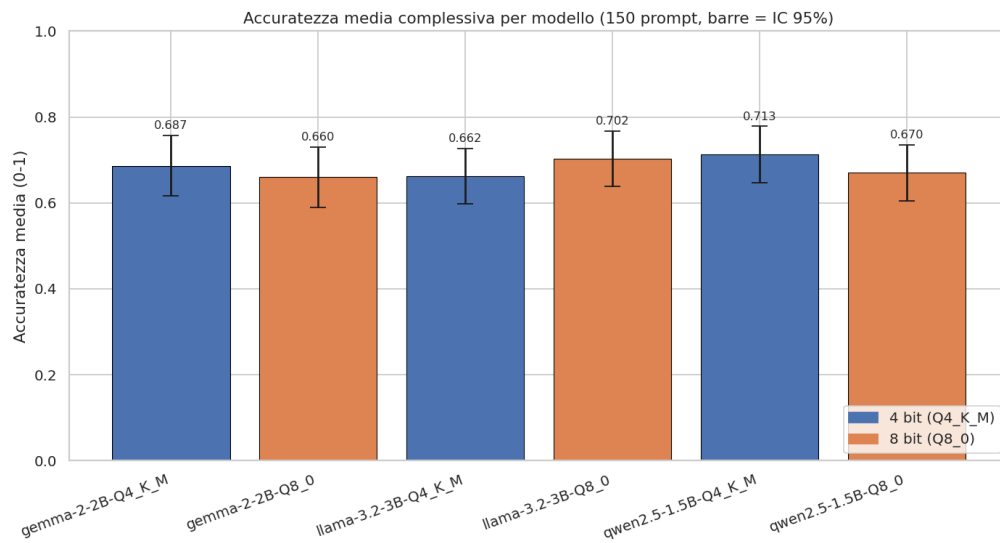


Figura 4.5: Accuratezza media complessiva per modello (Campagna A, 150 prompt), con barre d'errore pari all'intervallo di confidenza al 95%. All'interno di ogni famiglia gli intervalli delle due precisioni (4 e 8 bit) si sovrappongono ampiamente: le differenze di qualità non sono statisticamente significative.

4.1.3 Velocità di elaborazione

La Figura 4.6 riporta la velocità di elaborazione dei prompt (*prompt processing*) e di generazione (*generation*) in token al secondo. La generazione, che determina in larga parte la latenza percepita, varia da circa **5,1** a **17,9** token/s a seconda del modello: come atteso i modelli più piccoli (qwen2.5-1.5B, fino a 17,9 t/s) generano molto più rapidamente dei più grandi (Llama-3.2-3B e gemma-2-2B a 8 bit, intorno a 5,1 t/s). L'effetto della quantizzazione sulle due fasi è però *opposto*. In **generazione** — che produce un token alla volta ed è limitata dalla larghezza di banda della memoria — il 4 bit è più veloce dell'8 bit (ad esempio qwen2.5-1.5B: 17,9 contro 11,5 t/s), perché deve leggere circa metà dei byte per token. Nel **prompt processing**, invece, i token del prompt vengono elaborati insieme (prodotti matrice-matrice, con i pesi riutilizzati su molti token): la fase è limitata dal *calcolo* più che dalla memoria, e a pesare è il costo di dequantizzazione dello schema a 4 bit (Q4_K_M, più complesso del semplice Q8_0). Di conseguenza è l'**8 bit** a risultare più veloce nel prompt processing (qwen2.5-1.5B: 41,2 contro 31,8 t/s; gemma e Llama circa +10–13%). Poiché però la latenza complessiva è dominata dalla generazione (256 token prodotti a fronte di prompt brevi), nel complesso il 4 bit resta più rapido *end-to-end*.

Questo effetto si riflette direttamente sulla **latenza** di inferenza (Figura 4.7). A parità di modello e di parallelismo (2 thread), le varianti a **8 bit** risultano mediamente circa il **30% più lente** delle corrispondenti a 4 bit: l'aumento va da +13% per qwen2.5-1.5B (da 6,0 a 6,8 s) a +34% per gemma-2-2B (da 9,6 a 12,9 s) e +37% per Llama-3.2-3B (da 12,1 a 16,5 s). La ragione è la stessa che spiega il maggior consumo: gli 8 bit richiedono di leggere circa il doppio dei byte per ogni token generato e, su un dispositivo limitato dalla larghezza di banda della memoria, ciò allunga in proporzione i tempi di generazione. I 4 bit offrono quindi, oltre a un minor consumo, anche una latenza sensibilmente inferiore.

4.2 Effetto del grado di parallelismo (Campagna B)

Poiché il numero di thread influenza solo i tempi e i consumi dell'inferenza e non le risposte generate dal modello, in questa campagna la qualità non viene ridiscussa: essa è stata misurata in modo robusto nella Campagna A. Qui l'attenzione è rivolta all'effetto del parallelismo su latenza, potenza ed energia.

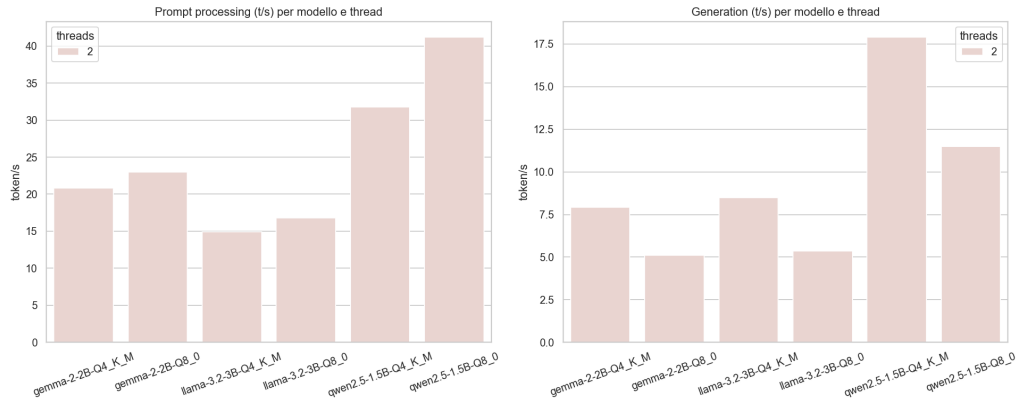


Figura 4.6: Velocità di elaborazione (prompt e generazione) per modello (Campagna A).

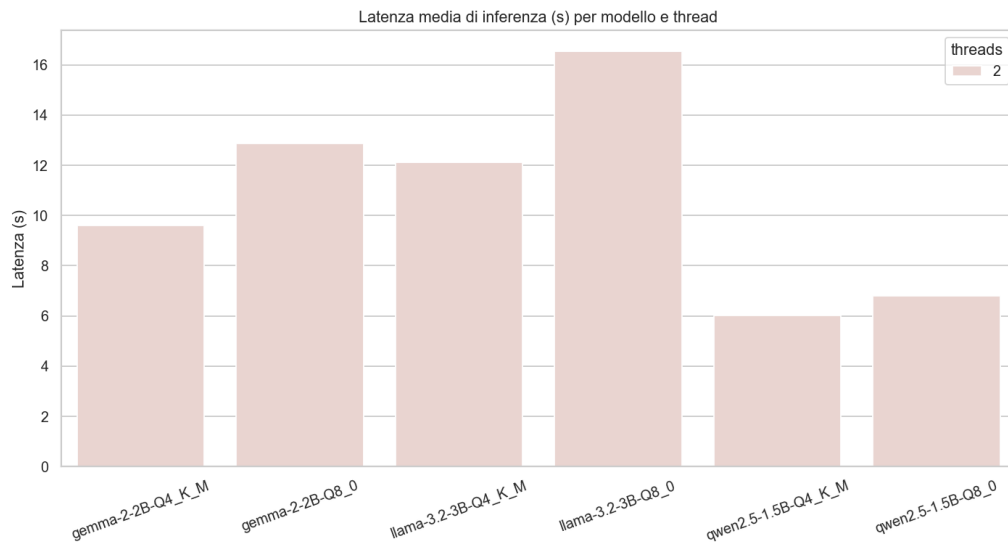


Figura 4.7: Latenza media di inferenza per modello, a 2 thread (Campagna A). A parità di famiglia, la variante a 8 bit (Q8_0) presenta una latenza superiore rispetto a quella a 4 bit (Q4_K_M).

4.2.1 Latenza

La Figura 4.8 mostra la latenza media di inferenza al variare del numero di thread (1, 2 e 4). Passando da 1 a 2 thread la latenza si riduce di circa **24%** (da 14,2 a 10,9 s in media), mentre l'ulteriore passaggio da 2 a 4 thread non porta alcun guadagno — anzi la latenza *peggiora* (circa +18%, risalendo a 12,8 s). Diversi benchmark sono limitati dalla larghezza di banda della memoria più che dal numero di core: il terzo e quarto core non accelerano la generazione e introducono contesa. Il valore ottimale di parallelismo risulta quindi **2 thread**.

Un dettaglio interessante riguarda il confronto tra le due precisioni *a 1 thread*: per ogni modello la latenza è più bassa con la variante a **8 bit** che con quella a 4 bit (ad esempio qwen2.5-1.5B: 7,6 s contro 9,6 s), in apparente contrasto con il fatto che i 4 bit debbano leggere meno dati. La spiegazione è ancora la distinzione tra carico limitato dalla *memoria* e carico limitato dal *calcolo*. Con un solo thread il bus di memoria non è saturo, quindi anche la generazione è più *compute-bound* che *memory-bound*: in questo regime il vantaggio del 4 bit (metà dei byte da leggere) è annullato dal suo maggior costo di **dequantizzazione** (lo schema Q4_K_M è più complesso del semplice Q8_0), tanto che la velocità di generazione delle due precisioni diventa praticamente identica (a 1 thread circa 7,7 t/s per il 4 bit contro 7,5 t/s per l'8 bit). A ciò si aggiunge la fase di *prompt processing*, anch'essa compute-bound e più rapida a 8 bit: il risultato netto è una latenza complessiva inferiore per l'8 bit a 1 thread. Passando a 2 o 4 thread, invece, i core saturano la banda di memoria e la generazione torna *memory-bound*: il 4 bit riprende un netto vantaggio (a 2 thread 11,5 contro 7,4 t/s) e, poiché la generazione domina la latenza complessiva, è di nuovo il 4 bit ad avere la latenza più bassa.

4.2.2 Potenza ed energia

All'aumentare del numero di thread la potenza media assorbita **cresce** (da circa 4,9 W a 1 thread a 7,0 W a 4 thread, Figura 4.9). L'energia netta per inferenza è **minima a 1 thread** (circa 40,6 J) e cresce con il parallelismo: solo lievemente da 1 a 2 thread (+10%, circa 44,8 J), ma nettamente a 4 thread (+28% rispetto a 2 thread, circa 57,5 J). In termini di sola energia il minimo si ha quindi a 1 thread, ma a fronte di una latenza molto più elevata; il miglior compromesso tra *time-to-answer* ed efficienza energetica è a **2 thread**, che riduce di circa un quarto la latenza con un modesto sovrapprezzo energetico, mentre a 4 thread si perde su entrambi i fronti.

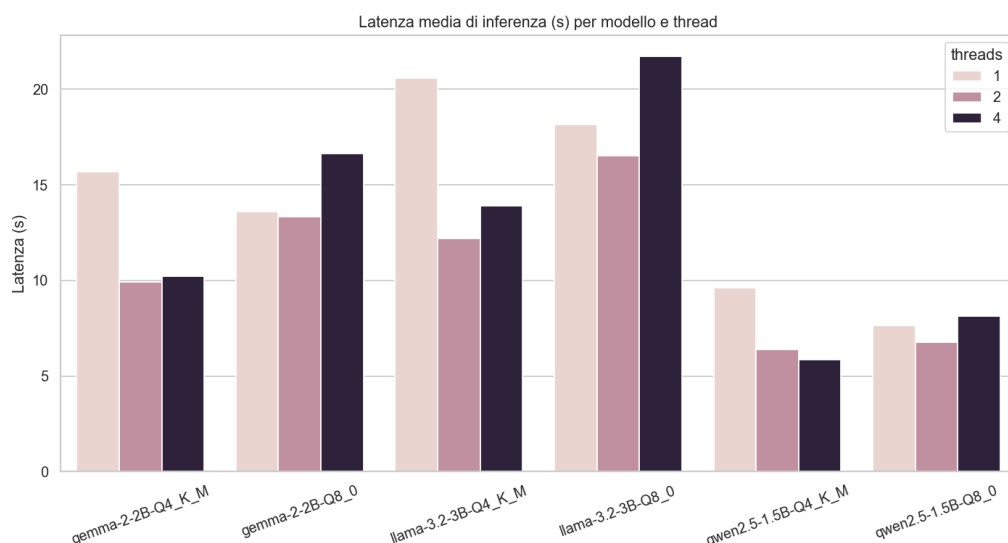


Figura 4.8: Latenza media di inferenza al variare del numero di thread (Campagna B).

Un dettaglio degno di nota riguarda il confronto tra le due precisioni al variare del parallelismo: la potenza media delle varianti a 4 e a 8 bit si *ribalta* passando da 1 a 4 thread. A **1 thread** le varianti a 8 bit assorbono circa 0,4 W in più delle corrispondenti a 4 bit, mentre a **4 thread** sono queste ultime a consumare di più (circa 0,65 W in più) — e il comportamento è uniforme in tutte e tre le famiglie. A prima vista il risultato sorprende — il modello a 4 bit, più leggero, dovrebbe consumare sempre meno — ma si spiega ricordando che la potenza istantanea misura *quanto i core stanno effettivamente calcolando*, e non la quantità totale di lavoro da svolgere: un core che calcola assorbe potenza, mentre un core fermo in *stallo* ad attendere i dati dalla memoria ne assorbe molta meno. Poiché il carico è limitato dalla **larghezza di banda della memoria**, è proprio la quota di tempo che i core trascorrono in stallo a determinare il ribaltamento. Con un solo thread il bus di memoria non è saturo e il modello a 8 bit — che deve leggere circa il doppio dei byte per token — tiene il core più occupato, assorbendo un po' di più. Con quattro thread, invece, i core competono per la stessa banda condivisa: il modello a 8 bit la satura e i core trascorrono più tempo in stallo (un core fermo consuma meno), mentre il modello a 4 bit, più leggero, li mantiene attivi nel calcolo, assorbendo così una potenza maggiore. Questo *non* penalizza i 4 bit: la potenza aggiuntiva si traduce in calcolo utile, tant'è che a 4 thread le varianti a 4 bit restano più veloci (latenza media 10,0 s contro 15,5 s) e più efficienti (47,7 J contro 67,4 J per inferenza).

Va precisato che questa spiegazione, per quanto coerente con i dati e con la natura *memory-bandwidth bound* del carico, resta un’**interpretazione**: le misure disponibili registrano la sola potenza *complessiva* della scheda (la somma dei dodici rami del PMIC), non l’andamento dei singoli rami. Una dimostrazione diretta richiederebbe di visualizzare separatamente i **dodici canali del PMIC** — in particolare confrontando i rami della memoria (*DDR_VDD2*, *DDR_VDDQ*) con quello del core del SoC (*VDD_CORE*) — così da osservare l’effettivo spostamento del consumo tra sottosistema di memoria e sottosistema di calcolo al variare del grado di parallelismo.

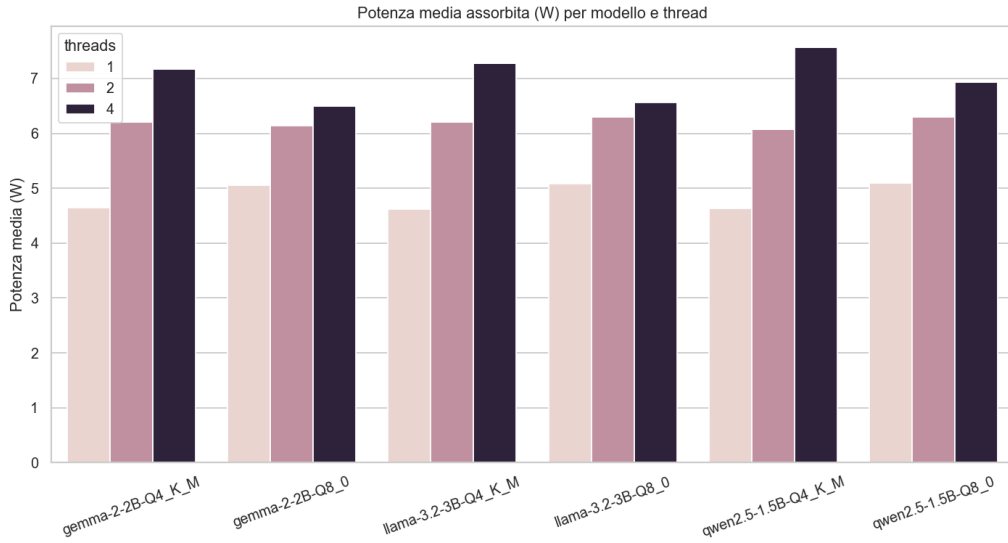


Figura 4.9: Potenza media assorbita al variare del numero di thread (Campagna B).

4.3 Frontiere di Pareto

Per individuare le configurazioni migliori senza fissare a priori un peso relativo tra gli obiettivi si ricorre al concetto di *frontiera di Pareto*: una configurazione si dice *dominata* se ne esiste un’altra migliore (o uguale) su entrambi gli obiettivi considerati, e strettamente migliore su almeno uno; le configurazioni *non dominate* costituiscono la frontiera, cioè l’insieme dei compromessi ottimali per cui non è possibile migliorare un obiettivo senza peggiorare l’altro. A differenza dello score composito, questa rappresentazione non richiede la scelta di pesi: evidenzia direttamente i candidati razionali e lascia al lettore

la decisione finale in base alle proprie priorità. Il criterio si applica sia al confronto tra i modelli (Campagna A) sia alla scelta del grado di parallelismo (Campagna B).

4.3.1 Confronto tra i modelli (Campagna A)

La Figura 4.10 riporta la frontiera nel piano **energia–qualità** (le configurazioni in basso a destra sono le migliori). Il risultato è particolarmente netto: **qwen2.5-1.5B Q4_K_M** unisce l'energia minima (24,6 J) a una qualità tra le più elevate; poiché nessun'altra configurazione la supera contemporaneamente su entrambi gli assi, essa risulta l'unico punto *non dominato* della frontiera. Va però ricordato che le differenze di qualità tra i modelli di testa rientrano nella variabilità di misura, per cui questa posizione dominante è trainata soprattutto dal consumo nettamente inferiore di qwen a 4 bit. Le configurazioni Llama-3.2-3B, in particolare, risultano nettamente *dominate*. La Figura 4.11 mostra l'analoga frontiera nel piano **latenza–qualità**, sempre per i sei modelli a 2 thread.

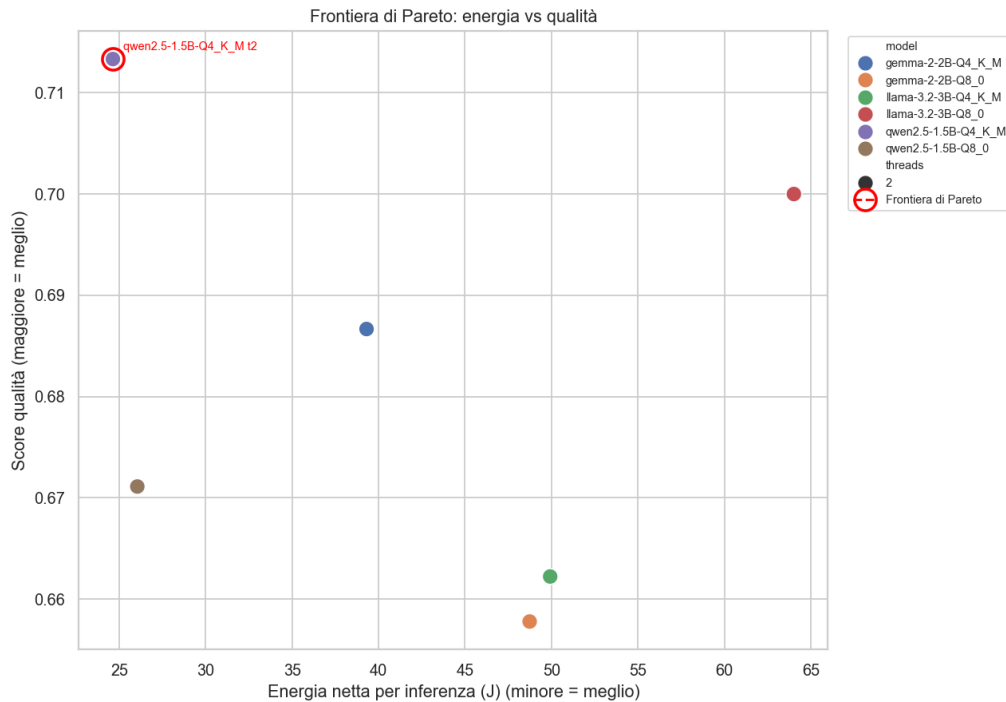


Figura 4.10: Frontiera di Pareto nel piano energia–qualità (Campagna A, i sei modelli a 2 thread; cerchiate in rosso le configurazioni non dominate).

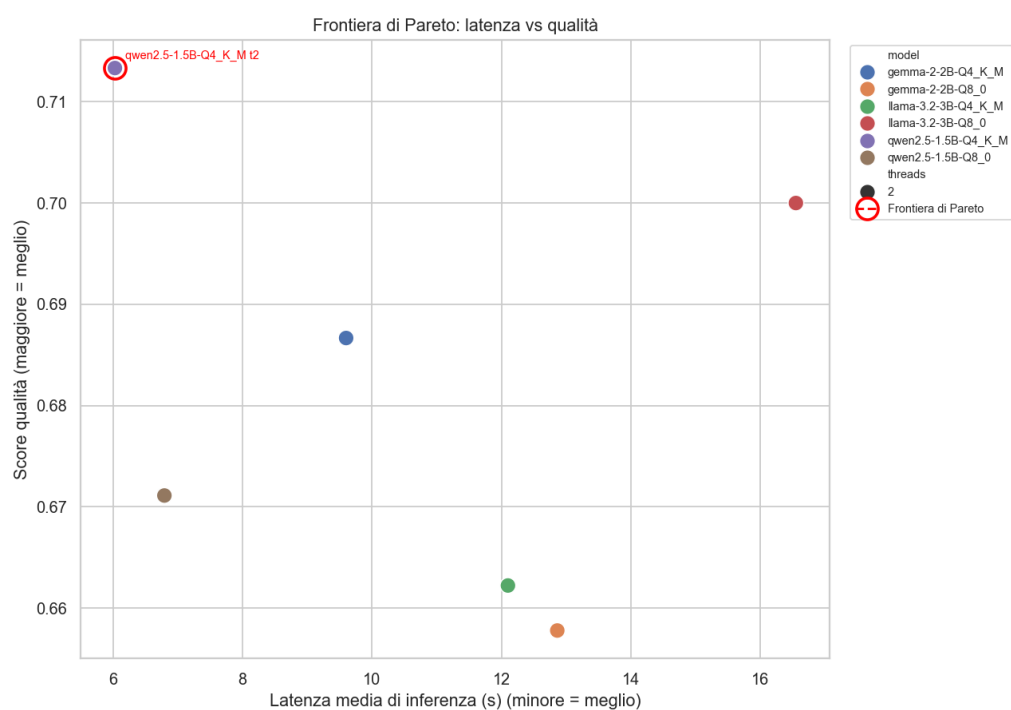


Figura 4.11: Frontiera di Pareto nel piano latenza–qualità (Campagna A, i sei modelli a 2 thread).

4.3.2 Scelta del grado di parallelismo (Campagna B)

Nella Campagna B la qualità non entra in gioco (il numero di thread non cambia le risposte): latenza ed energia sono due *costi*, entrambi da minimizzare. La Figura 4.12 riporta la frontiera di Pareto nel piano **latenza–energia** considerando tutte le configurazioni di modello e thread (le migliori sono in basso a sinistra). La frontiera è occupata *esclusivamente* da configurazioni **qwen2.5-1.5B**: qwen Q8 a 1 thread (energia minima, 23,8 J), qwen Q4 a 2 thread e qwen Q4 a 4 thread (latenza minima, 5,9 s); tutte le altre configurazioni — in particolare i modelli più grandi, a qualunque numero di thread — risultano *dominate*. Restrungendo l’analisi al singolo modello si ritrova quanto osservato sulla latenza: le configurazioni a **4 thread** sono quasi sempre dominate da quelle a 2 thread (consumano di più senza ridurre la latenza), e la scelta razionale del parallelismo resta tra **1 thread** (energia minima, ma latenza elevata) e **2 thread** (latenza minima, con un modesto sovrapprezzo energetico), con i **2 thread** come miglior compromesso complessivo.

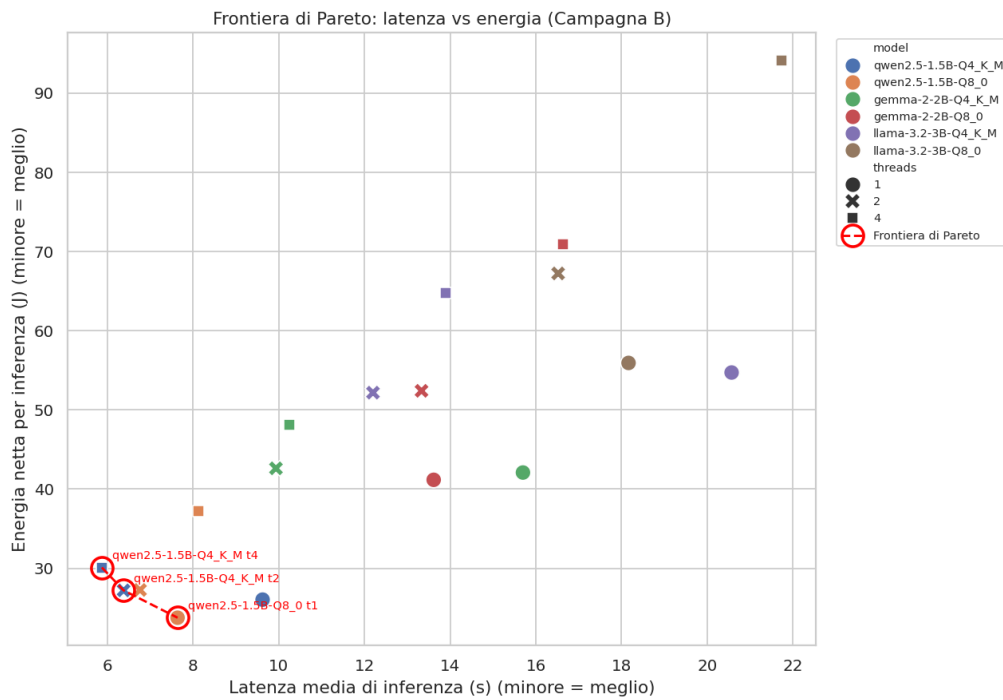


Figura 4.12: Frontiera di Pareto nel piano latenza–energia (Campagna B): tutte le configurazioni di modello e thread; la linea rossa collega le configurazioni non dominate. La frontiera è occupata dalle sole varianti qwen2.5-1.5B.

4.4 Sintesi: classifica complessiva

Per riassumere in un unico indicatore le tre dimensioni di analisi si definisce uno *score composito*. Le tre metriche di una configurazione — energia netta per inferenza E , latenza L e accuratezza media Q — hanno unità di misura e intervalli di valori diversi; per renderle confrontabili vengono normalizzate con una trasformazione *min-max* sull'insieme delle configurazioni:

$$\tilde{x} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \in [0, 1], \quad (4.1)$$

dove $x_{\min}$ e $x_{\max}$ sono il valore minimo e massimo di quella metrica tra tutte le configurazioni confrontate. Poiché per l'energia e la latenza sono preferibili valori *bassi*, mentre per la qualità è preferibile un valore *alto*, lo score composito C è definito come media pesata in cui i termini di energia e latenza vengono invertiti:

$$C = w_E (1 - \tilde{E}) + w_L (1 - \tilde{L}) + w_Q \tilde{Q}, \quad w_E + w_L + w_Q = 1, \quad (4.2)$$

con pesi $w_E = 0,4$ (energia), $w_L = 0,2$ (latenza) e $w_Q = 0,4$ (qualità). A un valore di C più alto corrisponde una configurazione complessivamente migliore.

La scelta dei pesi riflette le priorità dello studio: l'*efficienza energetica* e la *qualità delle risposte* sono considerate gli obiettivi primari e ricevono quindi lo stesso peso (0,4 ciascuno), mentre alla *latenza* è attribuito un peso inferiore (0,2), essendo un requisito meno stringente per molte applicazioni edge non strettamente interattive. Trattandosi di una scelta soggettiva, lo score composito va inteso come una sintesi orientativa e non come un giudizio assoluto: per questo motivo è affiancato dalla frontiera di Pareto (Figure 4.10 e 4.11), che individua le configurazioni ottimali *senza* richiedere alcuna scelta di pesi.

Lo score è calcolato sulle sei configurazioni della **Campagna A** (i sei modelli a 2 thread), coerentemente con l'obiettivo di confrontare i modelli a parità di grado di parallelismo. La Figura 4.13 riporta la relativa classifica. La configurazione complessivamente migliore risulta **qwen2.5-1.5B Q4_K_M a 2 thread** (score **1,00**), seguita da **qwen2.5-1.5B Q8_0** (0,67) e **gemma-2-2B Q4_K_M** (0,59): qwen a 4 bit ottiene il punteggio massimo perché unisce l'energia più bassa e la latenza più bassa a una qualità competitiva — il primato è trainato soprattutto dai vantaggi, robusti, su energia e latenza, dato che le differenze di qualità tra i modelli di testa rientrano nel rumore di misura —, mentre le configurazioni Llama-3.2-3B, penalizzate da consumo ed energia elevati, si collocano in fondo alla classifica nonostante la buona qualità sui task di ragionamento.

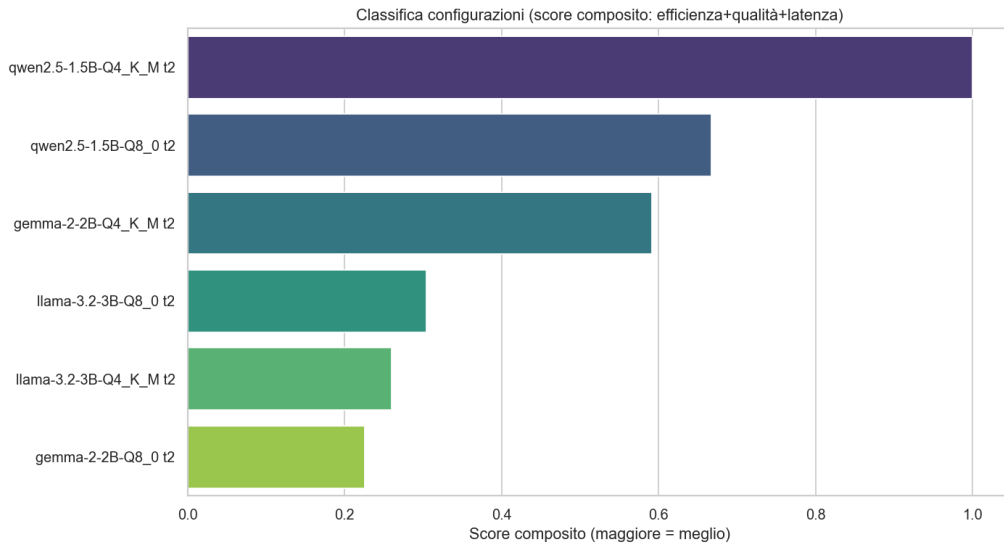


Figura 4.13: Classifica complessiva dei sei modelli secondo lo score composito (Campagna A, a 2 thread).

4.5 Comportamento termico

Durante entrambe le campagne la temperatura del SoC è stata monitorata dopo ogni inferenza. Grazie alla ventola di raffreddamento attiva — e, nella Campagna B, alle pause di raffreddamento tra le configurazioni — la temperatura si è mantenuta sotto la soglia di *throttling* ($\sim 80\text{--}85\text{ }^{\circ}\text{C}$): nella Campagna A (2 thread) il massimo è stato di **76,8** $^{\circ}\text{C}$, mentre nella Campagna B i picchi si sono avuti sulle configurazioni a 4 thread, comunque contenuti intorno agli **81** $^{\circ}\text{C}$ (Figura 4.14). In **nessuna** delle due campagne si è verificato throttling, per cui le misure di latenza ed energia non sono state falsate da riduzioni di frequenza indotte dal calore. Poiché in entrambe le campagne il funzionamento è avvenuto a piena frequenza, le misure restano inoltre *direttamente confrontabili*, nonostante la Campagna B adotti le pause di raffreddamento assenti nella Campagna A: la pausa agisce solo sulla temperatura di picco, che al di sotto della soglia di throttling non influenza né la latenza né l'energia.

Va tuttavia osservato che sessioni di misura molto prolungate, sommando il riscaldamento del SoC alla sollecitazione continua in lettura della scheda microSD, possono compromettere l'integrità dei file dei modelli memorizzati; per questo motivo la procedura adotta la verifica di integrità all'avvio e le pause di raffreddamento descritte nel capitolo precedente.

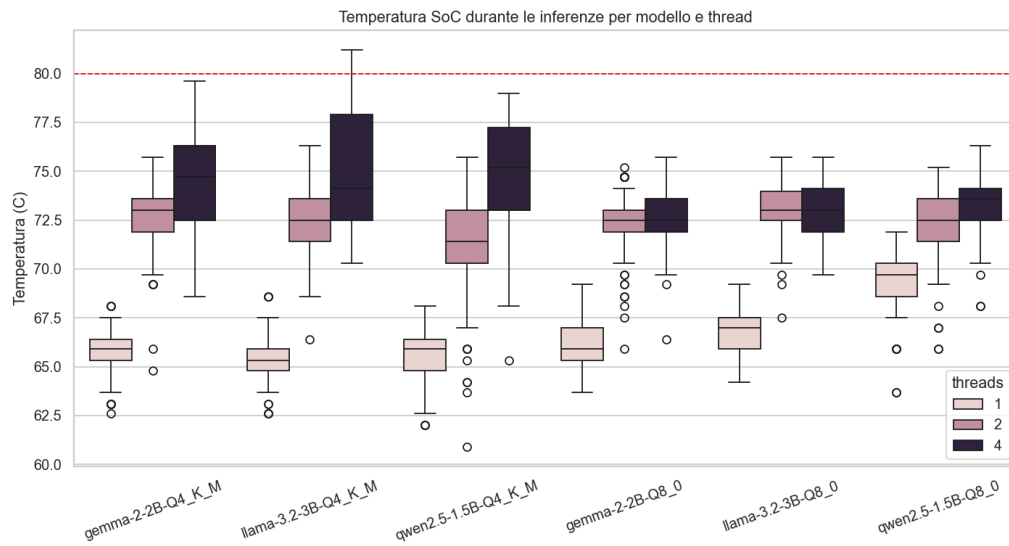


Figura 4.14: Distribuzione della temperatura del SoC durante le inferenze della Campagna B, che comprende le configurazioni a 4 thread, termicamente più impegnative.

Capitolo 5

Conclusioni

Questo lavoro ha valutato sperimentalmente il consumo energetico, la qualità delle risposte e la latenza di sei Large Language Model quantizzati eseguiti localmente su un Raspberry Pi 5, misurando il consumo direttamente dal PMIC della scheda. L'obiettivo era capire se e a quali condizioni modelli di questa taglia siano impiegabili in modo efficiente su un dispositivo dell'Internet delle Cose, e quale combinazione di modello, quantizzazione e grado di parallelismo offra il miglior compromesso.

5.1 Sintesi dei risultati

Rispetto alle tre dimensioni indagate, i risultati (Capitolo 4) possono essere riassunti come segue:

- **Efficienza energetica.** Il modello più efficiente per inferenza è risultato **qwen2.5-1.5B Q4_K_M**, con circa **24,6 J** per inferenza; in generale le varianti a 4 bit (**Q4_K_M**) si sono confermate più economiche delle corrispondenti a 8 bit (in media circa il 18% in meno).
- **Qualità delle risposte.** Le differenze di qualità tra i sei modelli sono risultate contenute (tutti tra 0,66 e 0,71) e non statisticamente significative (rientrano nell'incertezza di misura); in particolare, il passaggio da 8 a 4 bit ha comportato una differenza del tutto **trascurabile**, rendendo la quantizzazione più aggressiva a 4 bit **conveniente** nella maggior parte dei casi (stessa qualità, meno spazio occupato e meno energia).
- **Latenza e parallelismo.** L'aumento del numero di thread ha ridotto la latenza fino a **2 thread** (circa -24% rispetto a 1 thread), con **rendimenti nulli o negativi** oltre: a 4 thread la latenza non migliora

(anzi peggiora di circa il 18%) e l'energia peggiora (+28% rispetto a 2 thread). L'energia per inferenza è minima a 1 thread, ma a fronte di una latenza molto più alta, per cui i **2 thread** rappresentano il miglior compromesso. Anche la quantizzazione incide sulla latenza: a parità di modello e di thread, le varianti a **8 bit** sono risultate mediamente circa il **30% più lente** delle corrispondenti a 4 bit (da +13% per qwen a +37% per llama), per il maggior volume di dati da leggere per token — un ulteriore vantaggio dei 4 bit, che si aggiunge al minor consumo a fronte di una qualità sostanzialmente equivalente.

Complessivamente, secondo lo score composito che bilancia le tre metriche, la configurazione più equilibrata per un impiego su dispositivo a risorse limitate è **qwen2.5-1.5B Q4_K_M a 2 thread**.

5.2 Indicazioni pratiche

Dai risultati emergono alcune indicazioni operative per chi voglia eseguire un LLM su un dispositivo edge simile: privilegiare un **modello piccolo quantizzato a 4 bit** (qui qwen2.5-1.5B Q4_K_M ha offerto il miglior compromesso tra energia, latenza e qualità, risultando anche il più accurato in media; qualora però l'applicazione richieda spiccate capacità di ragionamento, il più grande Llama-3.2-3B eccelle sui compiti aritmetici e di ragionamento, ma a un costo energetico sensibilmente superiore), l'uso di **2 thread** come compromesso tra reattività ed efficienza, e la conferma che, con un adeguato raffreddamento, un Raspberry Pi 5 è in grado di sostenere l'inferenza di modelli fino a ~3 miliardi di parametri quantizzati senza incorrere in *throttling* termico.

5.3 Limiti del lavoro

I risultati vanno letti alla luce di alcuni limiti. Le misure di consumo sono acquisite internamente dal PMIC della scheda come somma dei prodotti tensione per corrente sui rami monitorati: rappresentano quindi il consumo interno della scheda e non quello alla presa di corrente, e non includono le perdite dell'alimentatore; per una stima del consumo “a parete” sarebbe necessaria una calibrazione dedicata sul singolo esemplare. La Campagna B, dedicata allo scaling sul parallelismo, adotta inoltre una **singola ripetizione** per configurazione (anziché le tre della Campagna A) per contenere i tempi di misura: la scelta è giustificata dal fatto che, sullo stesso insieme di 150 prompt, la variabilità tra prompt distinti domina quella tra ripetizioni,

ma comporta comunque una minore mediazione delle fluttuazioni casuali di misura rispetto alla Campagna A. Un ulteriore limite riguarda il supporto di archiviazione: l'esecuzione prolungata sotto carico può sollecitare la scheda microSD al punto da corromperne *silenziosamente* i file dei modelli — fenomeno mitigato dalla verifica di integrità `sha256` e dalle pause di raffreddamento adottate, ma che evidenzia la fragilità della microSD in un impiego così intensivo. Sul versante della qualità, inoltre, ogni benchmark è valutato su soli 30 prompt: i valori di accuratezza per singolo benchmark portano perciò un'incertezza dell'ordine di ± 9 punti percentuali e vanno interpretati a livello aggregato, mentre differenze di qualità inferiori a tale margine non sono statisticamente significative. Lo studio è infine limitato a un singolo esemplare di Raspberry Pi 5, a un unico motore di inferenza (`llama.cpp`) e a due soli formati di quantizzazione (`Q4_K_M` e `Q8_0`): l'estensione ad altre schede, backend o precisioni potrebbe modificare alcuni dei rapporti osservati.

5.4 Sviluppi futuri

Il lavoro può essere esteso in diverse direzioni. Sul piano delle misure, una calibrazione del PMIC rispetto a uno strumento di riferimento consentirebbe di riportare il consumo reale alla presa; la registrazione separata dei **dodici rami del PMIC** (anziché della sola somma) permetterebbe di attribuire il consumo ai singoli sottosistemi — memoria, core del SoC, I/O — e, ad esempio, di confermare direttamente l'origine del ribaltamento di potenza tra 4 e 8 bit osservato al variare del parallelismo nella Campagna B; un ulteriore ampliamento dei dataset e del numero di prompt aumenterebbe la robustezza statistica delle metriche di qualità e ne ridurrebbe l'incertezza per singolo benchmark; l'adozione di un supporto di archiviazione più affidabile — un'unità a stato solido collegata via USB oppure una microSD di classe industriale — eliminerebbe inoltre il rischio di corruzione dei file dei modelli osservato nelle campagne più lunghe. Sul piano sperimentale, sarebbe interessante includere ulteriori modelli e formati di quantizzazione (ad esempio precisioni inferiori a 4 bit), confrontare motori di inferenza alternativi, e valutare l'effetto di tecniche di gestione dinamica della frequenza e del numero di thread. Infine, l'analisi potrebbe essere estesa ad altri dispositivi edge per verificare la generalità delle conclusioni e per individuare il punto di equilibrio tra costo, consumo e qualità più adatto a ciascuna classe di applicazioni.

Capitolo 6

Ringraziamenti

Ringrazio il mio relatore, Prof. Alessio Vecchio, per la sua disponibilità e per avermi seguito durante lo svolgimento della tesi.

Il più grande ringraziamento va alla mia famiglia, mamma Patrizia, babbo Daniele e la mia sorellina più grande Alessia. Devo tutto a voi, vi voglio bene. Grazie.

Ai miei nipoti Simone e Davide. Spero che lo zio vi dia sempre quello che meritate di più mentre state crescendo. Questo mio traguardo è anche per voi.

Alla mia nonna Vera, che mi ha sempre sostenuto fino all'ultimo come un'altra mamma.

A tutti i miei zii, zie e cugini che mi hanno sempre dato speranza anche nei momenti bui.

Ai miei amici cardine di questa vita studiosa e non: Giovanni Ligato, Gabriele Shu, Matteo Perugini e Aurora Bracciotti. Siete in gamba dai, non mi lamento. Grazie per farmi sempre sentire in una botte di ferro.

Ai miei amici dell'università, Chiara, Eleonora P., Valentina, Eleonora B. (7agosto), Marta, Giulia, Martina, Alex, Francesco (Taki), Gabriele (Caio), Simone (il più forte), Daniele e Asia, con i quali condivido momenti di disperazione e di spensieratezza. Chi l'ha dura la vince, soprattutto con la mano che ci siamo dati a vicenda.

Ai miei amici di sempre, che nonostante i miei momenti di crisi, mi hanno sempre accolto con pazienza e tanto affetto. Un grosso grazie soprattutto ad Alessio Rosi, Gianmarco Picchi, Alessio Giovacchini, Alessio Poli e Leonardo Iannis.

Ai miei amici di studio da Tina, Luca, Ilaria, Giulia, Matilde V. e Matilde M. e altri, fino a Lorenzo Alessio, Niccolò (Fance), Luca, Martina, Sofia e altri. Troppi amici siete, ma non posso che ringraziarvi per avermi fatto stare bene in questi periodi di studio.

Ai miei amici dell'atletica, Manuel, Daniele, Alessio R., Alessio S., Gabriele, Simone, Leonardo, Mattia, Joao, Giulia e Lorenzo, che per quanto non sia riuscito a proseguire lo sport durante gli studi, vi ho sempre nel cuore. Porto sempre con me tutte le energie che ho sempre avuto durante la corsa insieme.

Tirando le somme con tutte queste persone citate e non, mi sento grato di avervi conosciuto in questi quasi 7 anni. Sono cresciuto molto grazie a voi, vi voglio bene.

Non so se si è capito,
Grazie.

Bibliografia

- [1] bartowski. gemma-2-2b-it-GGUF. <https://huggingface.co/bartowski/gemma-2-2b-it-GGUF>. Consultato il 3 luglio 2026.
- [2] bartowski. Llama-3.2-3B-Instruct-GGUF. <https://huggingface.co/bartowski/Llama-3.2-3B-Instruct-GGUF>. Consultato il 3 luglio 2026.
- [3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. https://huggingface.co/datasets/openai/openai_humaneval, 2021. Dataset HumanEval su Hugging Face. Consultato nel luglio 2026.
- [4] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. <https://huggingface.co/datasets/openai/gsm8k>, 2021. Dataset GSM8K su Hugging Face. Consultato nel luglio 2026.
- [5] Jan Fikar. RPi5-power: Simple script measuring the consumption of Raspberry Pi 5 in watts. <https://github.com/jfikar/RPi5-power>, 2024. Progetto open source. Consultato l'8 luglio 2026.
- [6] Erik Johannes Husom, Arda Goknil, Merve Astekin, Lwin Khin Shar, Andre Kåsen, Sagar Sen, Benedikt Andreas Mithassel, and Ahmet Soy-lu. Sustainable LLM inference for edge AI: Evaluating quantized LLMs for energy efficiency, output accuracy, and inference latency. *ACM Transactions on Internet of Things*, 6(4):28:1–28:35, 2025.
- [7] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. https://huggingface.co/datasets/truthful_qa, 2022. Dataset su Hugging Face (configurazione multiple_choice, MC1). Consultato nel luglio 2026.

- [8] Yuyi Mao, Xianghao Yu, Kaibin Huang, Ying-Jun Angela Zhang, and Jun Zhang. Green edge AI: A contemporary survey. *Proceedings of the IEEE*, 112(7):880–911, 2024.
- [9] Qwen Team. Qwen2.5-1.5B-Instruct-GGUF. <https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct-GGUF>. Consultato il 3 luglio 2026.
- [10] Raspberry Pi Ltd. Raspberry pi hardware — GPIO. <https://www.raspberrypi.com/documentation/computers/raspberry-pi.html#gpio>. Documentazione ufficiale. Consultato il 3 luglio 2026.
- [11] Raspberry Pi Ltd. Heating and cooling Raspberry Pi 5. <https://www.raspberrypi.com/news/heating-and-cooling-raspberry-pi-5/>, 2023. Documentazione ufficiale. Consultato il 2 luglio 2026.
- [12] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. Challenging BIG-Bench tasks and whether chain-of-thought can solve them. <https://huggingface.co/datasets/lukaemon/bbh>, 2022. Dataset BIG-Bench Hard su Hugging Face. Consultato nel luglio 2026.
- [13] Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. CommonsenseQA: A question answering challenge targeting commonsense knowledge. https://huggingface.co/datasets/tau/commonsense_qa, 2019. Dataset su Hugging Face. Consultato nel luglio 2026.